

Bootstrap practice date:23-02-2026

Problem1:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Products</title>

  <!-- Bootstrap 5 CSS CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>

<body class="bg-light">

  <div class="container mt-5">

    <!-- Section Title -->
    <h2 class="text-center mb-4 fw-bold">Our Products</h2>

    <!-- Responsive Product Grid -->
    <div class="row g-4">

      <!-- Product 1 -->
      <div class="col-lg-3 col-md-6 col-12">
        <div class="card shadow-sm border-primary h-100">
          <div class="card-body text-center">
            <h5 class="card-title text-primary">Product 1</h5>
            <p class="card-text">$100</p>
            <button class="btn btn-primary">Buy Now</button>
          </div>
        </div>
      </div>

      <!-- Product 2 -->
      <div class="col-lg-3 col-md-6 col-12">
        <div class="card shadow-sm border-success h-100">
          <div class="card-body text-center">
            <h5 class="card-title text-success">Product 2</h5>
            <p class="card-text">$150</p>
            <button class="btn btn-success">Buy Now</button>
          </div>
        </div>
      </div>

      <!-- Product 3 -->
      <div class="col-lg-3 col-md-6 col-12">
        <div class="card shadow-sm border-danger h-100">
          <div class="card-body text-center">
            <h5 class="card-title text-danger">Product 3</h5>
            <p class="card-text">$200</p>
            <button class="btn btn-danger">Buy Now</button>
          </div>
        </div>
      </div>

      <!-- Product 4 -->
```

```

<div class="col-lg-3 col-md-6 col-12">
  <div class="card shadow-sm border-warning h-100">
    <div class="card-body text-center">
      <h5 class="card-title text-warning">Product 4</h5>
      <p class="card-text">$250</p>
      <button class="btn btn-warning">Buy Now</button>
    </div>
  </div>
</div>
</div>
</div>
</div>
<!-- Bootstrap JS CDN -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

```

```

</body>

```

```

</html>

```

```

*****

```

Problem2: Bootstrap Navbar

```

<!DOCTYPE html>

```

```

<html lang="en">

```

```

<head>

```

```

  <meta charset="UTF-8">

```

```

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

```

```

  <title>MyShop Navbar</title>

```

```

  <!-- Bootstrap 5 CSS CDN -->

```

```

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">

```

```

</head>

```

```

<body>

```

```

  <!-- Navbar Start -->

```

```

  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">

```

```

    <div class="container">

```

```

      <!-- Brand -->

```

```

      <a class="navbar-brand fw-bold" href="#">MyShop</a>

```

```

      <!-- Mobile Toggle Button -->

```

```

      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">

```

```

        <span class="navbar-toggler-icon"></span>

```

```

      </button>

```

```

      <!-- Navbar Links -->

```

```

      <div class="collapse navbar-collapse" id="navbarNav">

```

```

        <!-- Left Menu -->

```

```

        <ul class="navbar-nav ms-auto">

```

```

          <li class="nav-item">

```

```

            <a class="nav-link active" href="#">Home</a>

```

```

          </li>

```

```

          <li class="nav-item">

```

```

            <a class="nav-link" href="#">Products</a>

```

```

          </li>

```

```

          <li class="nav-item">

```

```

            <a class="nav-link" href="#">About</a>

```

```

</li>

<li class="nav-item">
  <a class="nav-link" href="#">Contact</a>
</li>

<!-- Login Button -->
<li class="nav-item ms-lg-3">
  <a class="btn btn-warning" href="#">Login</a>
</li>

</ul>

</div>

</div>
</nav>
<!-- Navbar End -->

<!-- Bootstrap JS CDN -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

</body>
</html>
*****
Problem3: Customer Support
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Customer Support</title>

  <!-- Bootstrap 5 CSS CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>

<body class="bg-light">

  <div class="container mt-5">

    <!-- Customer Support Section -->
    <div class="p-5 mb-4 bg-white text-center border rounded shadow-sm">
      <h2 class="fw-bold mb-3">Customer Support</h2>
      <p class="mb-4">Need help? Access our support resources below.</p>

      <!-- Offcanvas Trigger Button -->
      <button class="btn btn-primary" type="button"
        data-bs-toggle="offcanvas"
        data-bs-target="#supportMenu">
        Support Menu
      </button>
    </div>

  </div>

  <!-- Offcanvas Menu -->
  <div class="offcanvas offcanvas-start" tabindex="-1" id="supportMenu">

```

```

<div class="offcanvas-header">
  <h5 class="offcanvas-title">Support Menu</h5>
  <button type="button" class="btn-close" data-bs-dismiss="offcanvas"></button>
</div>

<div class="offcanvas-body">
  <ul class="list-group">
    <li class="list-group-item">FAQs</li>
    <li class="list-group-item">Contact Support</li>
    <li class="list-group-item">Report Issue</li>
  </ul>
</div>
</div>

<!-- Bootstrap JS CDN (Required for Offcanvas) -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

</body>
</html>

```

Problem4: **Responsive Personal Portfolio Homepage**

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Personal Portfolio</title>

  <!-- Bootstrap 5 CSS CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>

<body class="bg-light">

  <!-- NAVBAR-->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
      <a class="navbar-brand fw-bold" href="#">MyPortfolio</a>

      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>

      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item"><a class="nav-link active" href="#">Home</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Skills</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Contact</a></li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- HERO SECTION -->
  <div class="container mt-5">
    <div class="row align-items-center">

```

```

<div class="col-lg-6 col-md-12 text-center text-lg-start">
  <h1 class="fw-bold">Hello, I'm Sony</h1>
  <p class="lead">A passionate fresher web developer skilled in building responsive websites using Bootstrap.</p>
  <button class="btn btn-primary me-2">Hire Me</button>
  <button class="btn btn-outline-secondary">Download CV</button>
</div>

<div class="col-lg-6 col-md-12 text-center mt-4 mt-lg-0">
  
</div>

</div>
</div>

<!-- SKILLS SECTION -->
<div class="container mt-5">
  <h2 class="text-center fw-bold mb-4">My Skills</h2>

  <div class="row g-4">

    <div class="col-lg-4 col-md-6 col-12">
      <div class="card shadow-sm h-100 text-center border-primary">
        <div class="card-body">
          <h5 class="card-title text-primary">HTML</h5>
          <p class="card-text">Strong foundation in semantic HTML structure.</p>
        </div>
      </div>
    </div>

    <div class="col-lg-4 col-md-6 col-12">
      <div class="card shadow-sm h-100 text-center border-success">
        <div class="card-body">
          <h5 class="card-title text-success">CSS</h5>
          <p class="card-text">Experience in styling responsive layouts.</p>
        </div>
      </div>
    </div>

    <div class="col-lg-4 col-md-6 col-12">
      <div class="card shadow-sm h-100 text-center border-danger">
        <div class="card-body">
          <h5 class="card-title text-danger">Bootstrap</h5>
          <p class="card-text">Build mobile-first responsive websites efficiently.</p>
        </div>
      </div>
    </div>

  </div>
</div>

<!-- CONTACT SECTION -->
<div class="container mt-5 mb-5">
  <div class="p-4 bg-dark text-white text-center rounded">
    <h3 class="fw-bold">Contact Me</h3>
    <p>Email: contact@email.com</p>
    <button class="btn btn-warning">Send Message</button>
  </div>
</div>

```

```

<!-- Bootstrap JS CDN -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

</body>
</html>
*****

Problem5: College Event Management Dashboard
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>College Event Dashboard</title>

  <!-- Bootstrap 5 CSS CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>

<body class="bg-light">

  <!-- NAVBAR -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
    <div class="container">
      <a class="navbar-brand fw-bold" href="#">College Events</a>

      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
        <span class="navbar-toggler-icon"></span>
      </button>

      <div class="collapse navbar-collapse" id="navbarNav">
        <ul class="navbar-nav ms-auto">
          <li class="nav-item"><a class="nav-link active" href="#">Dashboard</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Events</a></li>
          <li class="nav-item"><a class="nav-link" href="#">Register</a></li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- DASHBOARD CONTENT -->
  <div class="container mt-5">

    <div class="row g-4">

      <!-- Event List Section -->
      <div class="col-xl-8 col-lg-7 col-md-12">
        <div class="card shadow-sm">
          <div class="card-body">
            <div class="d-flex justify-content-between align-items-center mb-3">
              <h5 class="fw-bold mb-0">Event List</h5>
              <button class="btn btn-primary" data-bs-toggle="modal" data-bs-target="#addEventModal">
                Add Event
              </button>
            </div>

            <ul class="list-group">
              <li class="list-group-item">Tech Fest 2025</li>

```

```

        <li class="list-group-item">Cultural Day</li>
        <li class="list-group-item">Sports Meet</li>
    </ul>

</div>
</div>
</div>

<!-- Event Registration Form -->
<div class="col-xl-4 col-lg-5 col-md-12">
    <div class="card shadow-sm">
        <div class="card-body">
            <h5 class="fw-bold mb-3">Register for Event</h5>

            <form class="needs-validation" novalidate>
                <div class="mb-3">
                    <label class="form-label">Student Name</label>
                    <input type="text" class="form-control" required>
                    <div class="invalid-feedback">Please enter your name.</div>
                </div>

                <div class="mb-3">
                    <label class="form-label">Email</label>
                    <input type="email" class="form-control" required>
                    <div class="invalid-feedback">Enter valid email.</div>
                </div>

                <div class="mb-3">
                    <label class="form-label">Select Event</label>
                    <select class="form-select" required>
                        <option value="">Choose...</option>
                        <option>Tech Fest 2025</option>
                        <option>Cultural Day</option>
                        <option>Sports Meet</option>
                    </select>
                    <div class="invalid-feedback">Please select event.</div>
                </div>

                <button class="btn btn-success w-100" type="submit">Register</button>
            </form>

        </div>
    </div>
</div>

</div>

</div>

<!-- MODAL -->
<div class="modal fade" id="addEventModal" tabindex="-1">
    <div class="modal-dialog">
        <div class="modal-content">

            <div class="modal-header">
                <h5 class="modal-title">Add New Event</h5>
                <button type="button" class="btn-close" data-bs-dismiss="modal"></button>
            </div>

```

```

<div class="modal-body">
  <form>
    <div class="mb-3">
      <label class="form-label">Event Name</label>
      <input type="text" class="form-control">
    </div>

    <div class="mb-3">
      <label class="form-label">Event Date</label>
      <input type="date" class="form-control">
    </div>
  </form>
</div>

<div class="modal-footer">
  <button class="btn btn-secondary" data-bs-dismiss="modal">Close</button>
  <button class="btn btn-primary">Save Event</button>
</div>

</div>
</div>
</div>

<!-- Bootstrap JS Bundle CDN -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

```

```

</body>
</html>

```

```

*****

```

```

jQuery
problem1: Student Registration List

```

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Registration</title>

```

```

<!-- jQuery CDN -->
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

```

```

<style>
body {
  font-family: Arial, sans-serif;
  padding: 20px;
  background: #f5f5f5;
}

```

```

h2 {
  text-align: center;
}

```

```

.box {
  background: white;
  padding: 20px;
  max-width: 500px;
}

```

```

margin: auto;
border-radius: 10px;
box-shadow: 0px 0px 10px gray;
}

input {
width: 45%;
padding: 8px;
margin: 5px 0;
}

button {
padding: 8px 12px;
cursor: pointer;
}

table {
width: 100%;
margin-top: 15px;
border-collapse: collapse;
}

th, td {
border: 1px solid gray;
padding: 8px;
text-align: center;
}

tr {
cursor: pointer;
}

tr:hover {
background: tomato;
color: white;
}
</style>
</head>

<body>

<div class="box">
<h2>Student Registration</h2>

<!-- Form Inputs -->
<input type="text" id="nameInput" placeholder="Student Name">
<input type="text" id="rollInput" placeholder="Roll Number">
<button id="addBtn">Add Student</button>

<!-- Student Table -->
<table>
<thead>
<tr>
<th>Name</th>
<th>Roll No</th>
</tr>
</thead>
<tbody id="studentList"></tbody>

```

```

</table>

</div>

<script>
$(document).ready(function () {

    // Add Student
    $("#addBtn").click(function () {

        let name = $("#nameInput").val();
        let roll = $("#rollInput").val();

        if (name == "" || roll == "") {
            alert("Please fill all fields!");
        } else {
            $("#studentList").append(
                "<tr><td>" + name + "</td><td>" + roll + "</td></tr>"
            );

            $("#nameInput").val("");
            $("#rollInput").val("");
        }

    });

    // Remove student when row clicked
    $("#studentList").on("click", "tr", function () {
        $(this).remove();
    });

});
</script>

</body>
</html>
*****

```

Problem2: Live Character Counter

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Live Character Counter</title>

    <!-- jQuery CDN -->
    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

    <style>
    body {
        font-family: Arial, sans-serif;
        padding: 20px;
        background: #f5f5f5;
    }

    .box {
        background: white;
    }

```

```

padding: 20px;
max-width: 500px;
margin: auto;
border-radius: 10px;
box-shadow: 0px 0px 10px gray;
}

textarea {
width: 90%;
height: 100px;
padding: 10px;
font-size: 16px;
}

#count {
margin-top: 10px;
font-weight: bold;
}
</style>
</head>

<body>

<div class="box">
<h2>Live Character Counter</h2>

<textarea id="message" placeholder="Type your message..."></textarea>

<p id="count">Characters: 0</p>
<p id="warning"></p>
</div>

<script>
$(document).ready(function () {

    $("#message").keyup(function () {

        let textLength = $(this).val().length;

        $("#count").text("Characters: " + textLength);

        if (textLength > 100) {
            $("#warning").text("Limit exceeded!");
            $("#warning").css("color", "red");
        } else {
            $("#warning").text("");
        }

    });

});
</script>

</body>
</html>
*****
Problem3: Simple Shopping Cart Items
<!DOCTYPE html>

```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Shopping Cart Example</title>

  <!-- jQuery CDN -->
  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
    }

    ul {
      list-style: none;
      padding: 0;
    }

    li {
      background: lightgray;
      padding: 10px;
      margin: 5px 0;
      border-radius: 5px;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }

    .removeBtn {
      background: red;
      color: white;
      border: none;
      padding: 5px 10px;
      border-radius: 5px;
      cursor: pointer;
    }
  </style>
</head>

<body>

  <h2>Simple Shopping Cart</h2>

  <!-- Dropdown -->
  <select id="itemSelect">
    <option value="">Select Item</option>
    <option>Apple</option>
    <option>Mango</option>
    <option>Banana</option>
  </select>

  <button id="addBtn">Add to Cart</button>

  <h3>Cart Items:</h3>
  <ul id="cartList"></ul>
```

```
<h4>Total Items: <span id="total">0</span></h4>
```

```
<script>
```

```
$(document).ready(function () {
```

```
    let selectedItem = "";
```

```
    // When dropdown changes
```

```
    $("#itemSelect").change(function () {
```

```
        selectedItem = $(this).val();
```

```
    });
```

```
    // When Add button is clicked
```

```
    $("#addBtn").click(function () {
```

```
        if (selectedItem == "") {
```

```
            alert("Please select an item!");
```

```
        } else {
```

```
            // Append selected item
```

```
            $("#cartList").append(
```

```
                "<li>" + selectedItem +
```

```
                " <button class='removeBtn'>Remove</button></li>"
```

```
            );
```

```
            updateTotal();
```

```
        }
```

```
    });
```

```
    // Remove item when clicked (Dynamic)
```

```
    $("#cartList").on("click", ".removeBtn", function () {
```

```
        $(this).parent().remove();
```

```
        updateTotal();
```

```
    });
```

```
    // Function to update total count
```

```
    function updateTotal() {
```

```
        let count = $("#cartList li").length;
```

```
        $("#total").text(count);
```

```
    }
```

```
});
```

```
</script>
```

```
</body>
```

```
</html>
```

```

DROP TABLE IF EXISTS Patient;
CREATE TABLE Patient(
    PatID INT PRIMARY KEY,
    PatName VARCHAR(100) NOT NULL,
    PatNum VARCHAR(15)
);
DROP TABLE IF EXISTS Doctor;
CREATE TABLE Doctor(
    DoctID INT PRIMARY KEY,
    DoctName VARCHAR(100) NOT NULL,
    DocSp VARCHAR(100),
    Fees DECIMAL(10,2)
);
DROP TABLE IF EXISTS AppDet;
CREATE TABLE AppDet(
    AptID INT PRIMARY KEY,
    AppDate DATE,
    PatID INT,
    DoctID INT,
    FOREIGN KEY (PatID) REFERENCES Patient(PatID),
    FOREIGN KEY (DoctID) REFERENCES Doctor(DoctID)
);
--patient data
INSERT INTO Patient (PatID, PatName, PatNum) VALUES
(101, 'Sony', '9999'),
(102, 'vani', '8888'),
(103, 'Monika', '7777'),
(104, 'Parnika', '6666'),
(105, 'Dhanvik', '5555');

--doctor data
INSERT INTO Doctor (DoctID, DoctName, DocSp, Fees) VALUES
(10, 'Dr. Divya', 'Cardiology', 500.00),
(11, 'Dr. Dharani', 'Dermatology', 400.00),
(12, 'Dr. Rivansh', 'Neurology', 600.00),
(13, 'Dr. Devansh', 'Orthopedics', 450.00),
(14, 'Dr. Monisha', 'Pediatrics', 350.00);

--appointment
INSERT INTO AppDet (AptID, AppDate, PatID, DoctID) VALUES
(1, '2026-01-01', 101, 10),
(2, '2026-01-05', 102, 11),
(3, '2026-01-10', 101, 12),
(4, '2026-01-15', 103, 10),
(5, '2026-01-20', 104, 14),
(6, '2026-01-25', 105, 13),
(7, '2026-02-01', 101, 10);

-- to get data
SELECT Patient.PatName,
    Doctor.DoctName,
    Doctor.DocSp,
    AppDet.AppDate,
    Doctor.Fees
FROM AppDet
JOIN Patient ON AppDet.PatID = Patient.PatID
JOIN Doctor ON AppDet.DoctID = Doctor.DoctID;

```

--second question:

--- Department

DROP TABLE IF EXISTS Department;

CREATE TABLE Department(
 DeptID INT PRIMARY KEY,

 DeptName VARCHAR(30) NOT NULL

);

--Employees table

DROP TABLE IF EXISTS Employees;

CREATE TABLE Employees(
 EmpID INT PRIMARY KEY,

 Name VARCHAR(30) NOT NULL,
 Salary INT,

 DeptID INT ,
 FOREIGN KEY(DeptID) REFERENCES Department(DeptID)

);

---Project

DROP TABLE IF EXISTS Projects;

CREATE TABLE Projects(
 ProjectID INT PRIMARY KEY,

 ProjectName VARCHAR(30) NOT NULL,
 EmpID INT,

 DeptID INT,
 FOREIGN KEY(DeptID) REFERENCES Department(DeptID)

);

INSERT INTO Department (DeptID, DeptName) VALUES

(1, 'HR'),

(2, 'IT'),

(3, 'Finance');

INSERT INTO Employees (EmpID, Name, Salary, DeptID) VALUES

(101, 'Amit', 50000, 1),

(102, 'Priya', 65000, 2),

(103, 'Rohan', 70000, 2),

(104, 'Neha', 55000, 3),

(105, 'Kavya', 60000, 1);

INSERT INTO Projects (ProjectID, ProjectName, DeptID) VALUES

(201, 'Payroll System', 1),

(202, 'Website Development', 2),

(203, 'Mobile App', 2),

(204, 'Budget Analysis', 3);

SELECT Employees.EmpID, Employees.Name, Department.DeptName, Projects.ProjectName
FROM Department

JOIN Employees on Department.DeptID = Employees.DeptID

JOIN Projects on Department.DeptID=Projects.DeptID;

--filter,ORDER BY

```
SELECT Name, Salary
FROM Employees
WHERE Salary > 50000
ORDER BY Salary DESC;
```

--expression column

```
SELECT Name, Salary, Salary*12 AS AnnualSalary
FROM Employees
```

--QUESTION 3

```
DROP TABLE IF EXISTS Orders;
```

```
CREATE TABLE Orders(
    OrderID INT PRIMARY KEY,
    CustomerName VARCHAR(20) NOT NULL,
    ProductName VARCHAR(30),
    Category VARCHAR(30),
    Quantity INT,
    Price DECIMAL(10,2)
);
```

```
INSERT INTO Orders (OrderID, CustomerName, ProductName, Category, Quantity, Price) VALUES
(1, 'Divya', 'Laptop', 'Electronics', 1, 55000.00),
(2, 'Priya', 'Smartphone', 'Electronics', 2, 20000.00),
(3, 'Devansh', 'T-Shirt', 'Clothing', 3, 799.00),
(4, 'Riyash', 'Jeans', 'Clothing', 1, 1499.00),
(5, 'Kavya', 'Rice Bag', 'Grocery', 5, 1200.00),
(6, 'Arjun', 'Cooking Oil', 'Grocery', 2, 250.00),
(7, 'Meera', 'Headphones', 'Electronics', 1, 1999.00),
(8, 'Sneha', 'Jacket', 'Clothing', 1, 2499.00);
```

```
SELECT OrderID, Quantity, Price, Quantity * Price AS TotalAmount
FROM Orders;
```

```
SELECT Category, SUM(Quantity * Price) AS TotalSale
FROM Orders
GROUP BY Category
ORDER BY TotalSale DESC
LIMIT 1;
```

```
SELECT CustomerName, SUM(Quantity * Price) AS TotalSpending
FROM Orders
GROUP BY CustomerName
HAVING SUM(Quantity * Price) >
(
    SELECT AVG(CustomerTotal)
    FROM
    (
        SELECT SUM(Quantity * Price) AS CustomerTotal
        FROM Orders
        GROUP BY CustomerName
    ) AS AvgTable
);
```

```
SELECT Category, SUM(Quantity * Price) AS TotalSale
```

```
FROM Orders
GROUP BY Category
HAVING SUM(Quantity * Price) > 5000;
```

--problem4:

```
DROP TABLE IF EXISTS Batch1;
CREATE TABLE Batch1(
    Name VARCHAR(20)
);
DROP TABLE IF EXISTS Batch2;
CREATE TABLE Batch2(
    Name VARCHAR(20)
);
INSERT INTO Batch1(Name) Values('Ram'),
('Laxman'),
('Anju'),
('Siv'),
('parvati'),
('gana');
```

```
INSERT INTO Batch2(Name) Values('eswar'),
('ramu'),
('hanu'),
('ganga'),
('parvati'),
('gana');
```

```
SELECT Name FROM Batch1
INTERSECT
SELECT Name FROM Batch2;
```

```
SELECT Name FROM Batch1
UNION
SELECT Name FROM Batch2;
```

```
SELECT Name FROM Batch1
EXCEPT
SELECT Name FROM Batch2;
```

```
DROP TABLE IF EXISTS Fees;
CREATE TABLE Fees(
    StudentName VARCHAR(20),
    FeesPaid INT,
    CourseName VARCHAR(20)
);
INSERT INTO Fees(StudentName, FeesPaid, CourseName) Values('sony',4000,'python'),
('divya',5000,'C'),
('jersey',2500, 'Core Java');
```

```
SELECT StudentName, FeesPaid,
CASE
WHEN FeesPaid>=3000 THEN 'Paid'
WHEN FeesPaid<3000 THEN 'Due'
END AS State1
FROM Fees;
```

--Problem5:

```
DROP TABLE IF EXISTS Customers;
CREATE TABLE Customers(
    CustID INT PRIMARY KEY,
    CustName VARCHAR(30) NOT NULL,
    CustEmail VARCHAR(100) UNIQUE NOT NULL
);
DROP TABLE IF EXISTS Categories;
CREATE TABLE Categories(
    CatID INT PRIMARY KEY,
    CategoryName VARCHAR(40) NOT NULL
);
DROP TABLE IF EXISTS Products;
CREATE TABLE Products(
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(30) NOT NULL,
    Price DECIMAL(10,2) CHECK (Price >= 0),
    CatID INT,
    FOREIGN KEY (CatID) REFERENCES Categories(CatID)
);
DROP TABLE IF EXISTS Orders;
CREATE TABLE Orders(
    OrderID INT PRIMARY KEY,
    OrderDate DATE,
    CustID INT,
    FOREIGN KEY (CustID) REFERENCES Customers(CustID)
);
DROP TABLE IF EXISTS OrderDetails;
CREATE TABLE OrderDetails(
    OrderDetailID INT PRIMARY KEY,
    OrderID INT,
    ProductID INT,
    Quantity INT CHECK (Quantity > 0),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

INSERT INTO Customers VALUES
(1, 'Divya', 'divya@gmail.com'),
(2, 'Priya', 'priya@gmail.com'),
(3, 'Arjun', 'arjun@gmail.com'),
(4, 'Meera', 'meera@gmail.com'),
(5, 'Sneha', 'sneha@gmail.com');

INSERT INTO Categories VALUES
(1, 'Electronics'),
(2, 'Clothing'),
(3, 'Grocery');

INSERT INTO Products VALUES
(101, 'Laptop', 55000, 1),
(102, 'Smartphone', 20000, 1),
(103, 'T-Shirt', 799, 2),
(104, 'Jeans', 1499, 2),
(105, 'Rice Bag', 1200, 3);

INSERT INTO Orders VALUES
```

```
(201, '2026-01-01', 1),
(202, '2026-01-02', 2),
(203, '2026-01-03', 3),
(204, '2026-01-04', 4),
(205, '2026-01-05', 5),
(206, '2026-01-06', 1),
(207, '2026-01-07', 2),
(208, '2026-01-08', 3);
```

```
INSERT INTO OrderDetails VALUES
```

```
(1, 201, 101, 1),
(2, 202, 102, 2),
(3, 203, 103, 3),
(4, 204, 104, 1),
(5, 205, 105, 5),
(6, 206, 101, 1),
(7, 207, 103, 2),
(8, 208, 105, 4);
```

```
SELECT
```

```
o.OrderID,
c.CustName,
c.CustEmail,
p.ProductName,
cat.CategoryName,
p.Price,
od.Quantity,
(p.Price * od.Quantity) AS TotalAmount,
o.OrderDate
```

```
FROM Orders o
```

```
JOIN Customers c ON o.CustID = c.CustID
```

```
JOIN OrderDetails od ON o.OrderID = od.OrderID
```

```
JOIN Products p ON od.ProductID = p.ProductID
```

```
JOIN Categories cat ON p.CatID = cat.CatID;
```

```
--Morethan One Catergory
```

```
SELECT c.CustName
```

```
FROM Customers c
```

```
JOIN Orders o ON c.CustID = o.CustID
```

```
JOIN OrderDetails od ON o.OrderID = od.OrderID
```

```
JOIN Products p ON od.ProductID = p.ProductID
```

```
GROUP BY c.CustName
```

```
HAVING COUNT(DISTINCT p.CatID) > 1;
```

```
---most sold product
```

```
SELECT p.ProductName, SUM(od.Quantity) AS TotalQuantitySold
```

```
FROM OrderDetails od
```

```
JOIN Products p ON od.ProductID = p.ProductID
```

```
GROUP BY p.ProductName
```

```
ORDER BY TotalQuantitySold DESC
```

```
LIMIT 1;
```

```
--Delete never Ordered
```

```
DELETE FROM Products
```

```
WHERE ProductID NOT IN
```

```
(
```

```
SELECT DISTINCT ProductID
```

```
FROM OrderDetails
```

```
);
```

```
-----
```

```
--Problem6:
```

```

DROP TABLE IF EXISTS Customers;
CREATE TABLE Customers(
    CustomerID INT PRIMARY KEY,
    Name VARCHAR(30) NOT NULL,
    Phone VARCHAR(15) UNIQUE
);
DROP TABLE IF EXISTS Accounts;
CREATE TABLE Accounts(
    AccountID INT PRIMARY KEY,
    CustomerID INT,
    Balance DECIMAL(12,2) CHECK (Balance >= 0),
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);
DROP TABLE IF EXISTS Transactions;
CREATE TABLE Transactions(
    TransactionID INT PRIMARY KEY,
    AccountID INT,
    Type VARCHAR(10),
    Amount DECIMAL(10,2) CHECK (Amount > 0),
    Date DATE,
    FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)
);

```

```

INSERT INTO Customers VALUES
(1,'Divya','9876543210'),
(2,'Priya','9876543211'),
(3,'Arjun','9876543212'),
(4,'Meera','9876543213'),
(5,'Sneha','9876543214');

```

```

INSERT INTO Accounts VALUES
(101,1,50000),
(102,2,30000),
(103,3,45000),
(104,4,25000),
(105,5,60000),
(106,1,15000);

```

```

INSERT INTO Transactions VALUES
(1,101,'Credit',10000,'2026-01-01'),
(2,101,'Debit',5000,'2026-01-02'),
(3,102,'Credit',8000,'2026-01-03'),
(4,103,'Debit',7000,'2026-01-04'),
(5,104,'Credit',6000,'2026-01-05'),
(6,105,'Debit',4000,'2026-01-06'),
(7,106,'Credit',3000,'2026-01-07'),
(8,102,'Debit',2000,'2026-01-08'),
(9,103,'Credit',9000,'2026-01-09'),
(10,105,'Credit',5000,'2026-01-10');

```

```

--- calculate TotalCredits
SELECT AccountID,
    SUM(Amount) AS TotalCredits
FROM Transactions
WHERE Type = 'Credit'
GROUP BY AccountID;

```

--calculated credits, debits,balance using CASE

```

SELECT AccountID,
       SUM(CASE WHEN Type = 'Credit' THEN Amount ELSE 0 END) AS TotalCredits,
       SUM(CASE WHEN Type = 'Debit' THEN Amount ELSE 0 END) AS TotalDebits,
       SUM(CASE
           WHEN Type = 'Credit' THEN Amount
           WHEN Type = 'Debit' THEN -Amount
           ELSE 0
       END) AS CurrentBalance
FROM Transactions
GROUP BY AccountID;

```

```

SELECT
    c.Name AS CustomerName,
    a.AccountID,
    SUM(CASE WHEN t.Type = 'Credit' THEN t.Amount ELSE 0 END) AS TotalCredits,
    SUM(CASE WHEN t.Type = 'Debit' THEN t.Amount ELSE 0 END) AS TotalDebits,
    SUM(CASE
        WHEN t.Type = 'Credit' THEN t.Amount
        WHEN t.Type = 'Debit' THEN -t.Amount
        ELSE 0
    END) AS CurrentBalance
FROM Customers c
JOIN Accounts a ON c.CustomerID = a.CustomerID
LEFT JOIN Transactions t ON a.AccountID = t.AccountID
GROUP BY c.Name, a.AccountID;

```

```

SELECT DISTINCT c.Name
FROM Customers c
JOIN Accounts a ON c.CustomerID = a.CustomerID
JOIN Transactions t ON a.AccountID = t.AccountID
WHERE t.Amount >
    (SELECT AVG(Amount) FROM Transactions);

```

```

-- updating account by expression
SELECT * FROM Accounts;

```

```

UPDATE Accounts
SET Balance = Balance * 1.02
WHERE Balance > 100000;

```

```

SELECT * FROM Accounts;
--delete balance 0 and no transaction
DELETE FROM Accounts
WHERE Balance = 0
AND AccountID NOT IN
(
    SELECT AccountID
    FROM Transactions
);

```

--- problem7:

```

DROP TABLE IF EXISTS Departments;
CREATE TABLE Departments(
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL
);

```

```

DROP TABLE IF EXISTS Students;
CREATE TABLE Students(
    StudentID INT PRIMARY KEY,

```

```

    Name VARCHAR(50) NOT NULL,
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
DROP TABLE IF EXISTS Marks;
CREATE TABLE Marks(
    StudentID INT,
    Subject VARCHAR(50),
    Marks INT CHECK (Marks >= 0 AND Marks <= 100),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
INSERT INTO Departments VALUES
(1, 'Computer Science'),
(2, 'Mechanical'),
(3, 'Commerce');
INSERT INTO Students VALUES
(101, 'Divya', 1),
(102, 'Priya', 1),
(103, 'Arjun', 2),
(104, 'Meera', 2),
(105, 'Sneha', 3),
(106, 'Rahul', 3),
(107, 'Kavya', 1),
(108, 'Rohan', 2);
INSERT INTO Marks VALUES
(101, 'Maths', 85),
(101, 'DBMS', 78),

(102, 'Maths', 92),
(102, 'DBMS', 88),

(103, 'Mechanics', 75),
(103, 'Thermodynamics', 80),

(104, 'Mechanics', 68),
(104, 'Thermodynamics', 72),

(105, 'Accounts', 90),
(105, 'Economics', 84),

(106, 'Accounts', 70),
(106, 'Economics', 65),

(107, 'Maths', 95),
(107, 'DBMS', 89),

(108, 'Mechanics', 60),
(108, 'Thermodynamics', 55);
SELECT s.StudentID,
       s.Name,
       SUM(m.Marks) AS TotalMarks
FROM Students s
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY s.StudentID, s.Name;
--case use
SELECT s.StudentID,
       s.Name,
       SUM(m.Marks) AS TotalMarks,

```

```

CASE
    WHEN SUM(m.Marks) >= 90 THEN 'A'
    WHEN SUM(m.Marks) >= 75 THEN 'B'
    WHEN SUM(m.Marks) >= 60 THEN 'C'
    ELSE 'Fail'
END AS Grade
FROM Students s
JOIN Marks m ON s.StudentID = m.StudentID
GROUP BY s.StudentID, s.Name;
--
--problem8:

DROP TABLE IF EXISTS Suppliers;
CREATE TABLE Suppliers(
    SupplierID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL
);
DROP TABLE IF EXISTS Products;
CREATE TABLE Products(
    ProductID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    SupplierID INT,
    StockQty INT CHECK (StockQty >= 0),
    FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID)
);
DROP TABLE IF EXISTS Orders;
CREATE TABLE Orders(
    OrderID INT PRIMARY KEY,
    ProductID INT,
    QuantityOrdered INT CHECK (QuantityOrdered > 0),
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
INSERT INTO Suppliers VALUES
(1,'TechCorp'),
(2,'FreshFoods'),
(3,'ClothMart'),
(4,'HomeNeeds'),
(5,'GlobalSupply');
INSERT INTO Products VALUES
(101,'Laptop',1,50),
(102,'Mouse',1,100),
(103,'Rice Bag',2,200),
(104,'T-Shirt',3,150),
(105,'Chair',4,80),
(106,'Table',4,40),
(107,'Oil Can',2,120),
(108,'Jacket',3,60);
INSERT INTO Orders VALUES
(1,101,5),
(2,102,10),
(3,103,20),
(4,104,15),
(5,105,8),
(6,106,4),
(7,101,3),
(8,103,25),
(9,107,12),
(10,108,6);

```

```

--- morethan one product
SELECT SupplierID, Name
FROM Suppliers
WHERE SupplierID IN
(
    SELECT SupplierID
    FROM Products
    GROUP BY SupplierID
    HAVING COUNT(ProductID) > 1
);
-- stock less than ordered quantity
SELECT p.ProductID,
       p.Name,
       p.StockQty,
       SUM(o.QuantityOrdered) AS TotalOrdered
FROM Products p
JOIN Orders o ON p.ProductID = o.ProductID
GROUP BY p.ProductID, p.Name, p.StockQty
HAVING p.StockQty < SUM(o.QuantityOrdered);
--using leftjoin shows never supplied
SELECT s.SupplierID, s.Name
FROM Suppliers s
LEFT JOIN Products p
    ON s.SupplierID = p.SupplierID
WHERE p.ProductID IS NULL;
-- increase stock by 50
UPDATE Products
SET StockQty = StockQty + 50
WHERE StockQty < 100;
--removes suppliers
DELETE FROM Suppliers
WHERE NOT EXISTS
(
    SELECT 1
    FROM Products p
    WHERE p.SupplierID = Suppliers.SupplierID
);
--use case with stockqty
SELECT ProductID,
       Name,
       StockQty,
       CASE
           WHEN StockQty >= 500 THEN 'High Stock'
           WHEN StockQty BETWEEN 200 AND 499 THEN 'Medium Stock'
           ELSE 'Low Stock'
       END AS StockStatus
FROM Products;

```

problem9:

```

CREATE TABLE Users(
    UserID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    SubscriptionType VARCHAR(20)
    CHECK (SubscriptionType IN ('Free','Basic','Premium'))
);

```

```

CREATE TABLE Movies(
    MovieID INT PRIMARY KEY,

```

```

    Title VARCHAR(100) NOT NULL,
    Genre VARCHAR(30) NOT NULL
);

CREATE TABLE WatchHistory(
    UserID INT,
    MovieID INT,
    WatchTime INT CHECK (WatchTime >= 0),
    FOREIGN KEY (UserID) REFERENCES Users(UserID),
    FOREIGN KEY (MovieID) REFERENCES Movies(MovieID)
);

```

```

INSERT INTO Users VALUES
(1,'Divya','Premium'),
(2,'Priya','Basic'),
(3,'Arjun','Free'),
(4,'Meera','Premium'),
(5,'Sneha','Basic');
INSERT INTO Movies VALUES
(101,'Inception','Sci-Fi'),
(102,'Titanic','Romance'),
(103,'Avengers','Action'),
(104,'Joker','Drama'),
(105,'Frozen','Animation'),
(106,'Interstellar','Sci-Fi');
INSERT INTO WatchHistory VALUES

```

```

(1,101,120),
(1,103,150),
(2,102,110),
(2,105,95),
(3,103,80),
(3,104,100),
(4,106,130),
(4,101,140),
(5,105,90),
(5,102,105);
--1
SELECT m.MovieID,
       m.Title,
       SUM(w.WatchTime) AS TotalWatchTime
FROM WatchHistory w
JOIN Movies m ON w.MovieID = m.MovieID
GROUP BY m.MovieID, m.Title
ORDER BY TotalWatchTime DESC
LIMIT 1;

```

```

--2
SELECT u.UserID,
       u.Name,
       COUNT(DISTINCT m.Genre) AS GenreCount
FROM WatchHistory w
JOIN Movies m ON w.MovieID = m.MovieID
JOIN Users u ON w.UserID = u.UserID
GROUP BY u.UserID, u.Name
HAVING COUNT(DISTINCT m.Genre) > 2;
--3
SELECT Genre,
       SUM(TotalWatchTime) AS GenreTotalWatchTime
FROM
(

```

```

SELECT m.Genre,
       w.WatchTime AS TotalWatchTime
FROM WatchHistory w
JOIN Movies m ON w.MovieID = m.MovieID
) AS T
GROUP BY Genre;
--4
UPDATE Users
SET SubscriptionType = 'Premium'
WHERE UserID IN
(
  SELECT UserID
  FROM
  (
    SELECT UserID,
           SUM(WatchTime) AS TotalWatchTime
    FROM WatchHistory
    GROUP BY UserID
    HAVING SUM(WatchTime) >
    (
      SELECT AVG(UserTotal)
      FROM
      (
        SELECT SUM(WatchTime) AS UserTotal
        FROM WatchHistory
        GROUP BY UserID
      ) AS AvgTable
    )
  ) AS EligibleUsers
);
--5
SELECT m.MovieID, m.Title
FROM Movies m
WHERE NOT EXISTS
(
  SELECT 1
  FROM WatchHistory w
  WHERE w.MovieID = m.MovieID
);

```

problem10:

```

DROP TABLE IF EXISTS Performance;
DROP TABLE IF EXISTS Employees;
DROP TABLE IF EXISTS Departments;

```

```

CREATE TABLE Departments(
  DeptID INT PRIMARY KEY,
  DeptName VARCHAR(50) NOT NULL
);

```

```

CREATE TABLE Employees(
  EmpID INT PRIMARY KEY,
  Name VARCHAR(50) NOT NULL,
  Salary DECIMAL(10,2) CHECK (Salary >= 0),
  DeptID INT,
  FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);

```

```

CREATE TABLE Performance(
    EmpID INT PRIMARY KEY,
    Rating INT CHECK (Rating BETWEEN 1 AND 5),
    FOREIGN KEY (EmpID) REFERENCES Employees(EmpID)
);

```

```

INSERT INTO Departments VALUES

```

```

(1,'IT'),
(2,'HR'),
(3,'Finance');

```

```

INSERT INTO Employees VALUES

```

```

(101,'Divya',60000,1),
(102,'Priya',45000,2),
(103,'Arjun',75000,1),
(104,'Meera',50000,3),
(105,'Sneha',40000,2),
(106,'Rahul',90000,1),
(107,'Kavya',55000,3),
(108,'Rohan',65000,1);

```

```

INSERT INTO Performance VALUES (101,5);

```

```

INSERT INTO Performance VALUES (102,3);

```

```

INSERT INTO Performance VALUES (103,4);

```

```

INSERT INTO Performance VALUES (104,2);

```

```

INSERT INTO Performance VALUES (105,1);

```

```

UPDATE Employees

```

```

SET Salary = Salary * 1.10

```

```

WHERE EmpID IN

```

```

(
    SELECT EmpID
    FROM Performance
    WHERE Rating >= 4

```

```

);

```

```

---case

```

```

SELECT e.Name,
       d.DeptName,
       e.Salary,
       p.Rating,
       CASE
           WHEN p.Rating = 5 THEN 'Excellent'
           WHEN p.Rating = 4 THEN 'Very Good'
           WHEN p.Rating = 3 THEN 'Good'
           WHEN p.Rating = 2 THEN 'Average'
           ELSE 'Poor'
       END AS PerformanceLevel
FROM Employees e

```

```

JOIN Departments d ON e.DeptID = d.DeptID

```

```

JOIN Performance p ON e.EmpID = p.EmpID;

```

```

-----

```

```

SELECT e.Name,
       d.DeptName,
       e.Salary,
       p.Rating,
       CASE
           WHEN p.Rating = 5 THEN 'Excellent'
           WHEN p.Rating = 4 THEN 'Good'
           WHEN p.Rating = 3 THEN 'Average'
           ELSE 'Poor'
       END AS PerformanceLevel

```

```
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
JOIN Performance p ON e.EmpID = p.EmpID;
```

```
SELECT d.DeptName,
       AVG(e.Salary) AS AvgSalary
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
GROUP BY d.DeptName
HAVING AVG(e.Salary) =
(
  SELECT MAX(DeptAvg)
  FROM
  (
    SELECT AVG(Salary) AS DeptAvg
    FROM Employees
    GROUP BY DeptID
  ) AS T
);
```

```
SELECT e.Name,
       e.Salary,
       e.DeptID
FROM Employees e
WHERE e.Salary >
(
  SELECT AVG(Salary)
  FROM Employees
  WHERE DeptID = e.DeptID
);
```

```
DELETE FROM Employees
WHERE EmpID IN
(
  SELECT EmpID
  FROM Performance
  WHERE Rating < 2
);
```

11)project-11

- create table

```
CREATE TABLE Transactions (
  TransactionID INT PRIMARY KEY,
  UserID INT,
  Amount DECIMAL(10,2),
  Location VARCHAR(50),
  Date DATE
);
```

-- insert sample data

```
INSERT INTO Transactions VALUES
(1, 101, 5000, 'Chennai', '2024-01-10'),
(2, 101, 7000, 'Chennai', '2024-01-10'),
(3, 101, 9000, 'Chennai', '2024-01-10'),
(4, 101, 11000, 'Chennai', '2024-01-10'),
(5, 102, 150000, 'Mumbai', '2024-02-12'),
(6, 102, 2000, 'Delhi', '2024-02-13'),
(7, 103, 3000, 'Bangalore', '2018-03-15'),
(8, 103, 4000, 'Bangalore', '2017-04-20'),
```

```
(9, 104, 250000, 'Hyderabad', '2024-05-01'),
(10, 104, 260000, 'Hyderabad', '2024-05-02');
```

```
-- more than 3 transactions in one day
SELECT UserID, Date, COUNT(*) AS TotalTransactions
FROM Transactions
GROUP BY UserID, Date
HAVING COUNT(*) > 3;
```

```
-- users with multiple locations
SELECT UserID, COUNT(DISTINCT Location) AS LocationCount
FROM Transactions
GROUP BY UserID
HAVING COUNT(DISTINCT Location) > 1;
```

```
-- show risk level using correlated subquery
SELECT
    t.TransactionID,
    t.UserID,
    t.Amount,
    t.Location,
    CASE
        WHEN t.Amount > 100000 THEN 'High Risk'
        WHEN (
            SELECT COUNT(DISTINCT t2.Location)
            FROM Transactions t2
            WHERE t2.UserID = t.UserID
        ) > 1 THEN 'Location Risk'
        ELSE 'Normal'
    END AS RiskStatus
FROM Transactions t;
```

```
-- delete transactions older than 5 years
DELETE FROM Transactions
WHERE Date < DATE('now', '-5 years');
```

```
Project1:
PRAGMA foreign_keys = ON;
```

```
-- table Patients
DROP TABLE IF EXISTS Patients;
```

```
CREATE TABLE Patients (
    PatientID INTEGER PRIMARY KEY,
    FullName TEXT NOT NULL,
    DOB DATE NOT NULL,
    Gender TEXT CHECK (Gender IN ('Male', 'Female', 'Other')),
    Phone TEXT UNIQUE,
    Email TEXT UNIQUE,
    Address TEXT
);
```

```
-- table Specializations
DROP TABLE IF EXISTS Specializations;
```

```
CREATE TABLE Specializations (
    SpecializationID INTEGER PRIMARY KEY,
```

```

    SpecializationName TEXT UNIQUE NOT NULL
);

-- table Doctors
DROP TABLE IF EXISTS Doctors;

CREATE TABLE Doctors (
    DoctorID INTEGER PRIMARY KEY,
    FullName TEXT NOT NULL,
    Phone TEXT UNIQUE,
    Salary REAL NOT NULL DEFAULT 0,
    SpecializationID INTEGER NOT NULL,
    FOREIGN KEY (SpecializationID) REFERENCES Specializations(SpecializationID)
);

-- table Appointments
DROP TABLE IF EXISTS Appointments;

CREATE TABLE Appointments (
    AppointmentID INTEGER PRIMARY KEY,
    PatientID INTEGER NOT NULL,
    DoctorID INTEGER NOT NULL,
    AppointmentDate DATE NOT NULL,
    Status TEXT CHECK (Status IN ('Scheduled', 'Completed', 'Cancelled')),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)
);

-- table Treatments
DROP TABLE IF EXISTS Treatments;

CREATE TABLE Treatments (
    TreatmentID INTEGER PRIMARY KEY,
    AppointmentID INTEGER NOT NULL,
    TreatmentName TEXT NOT NULL,
    Cost REAL NOT NULL CHECK (Cost > 0),
    FOREIGN KEY (AppointmentID) REFERENCES Appointments(AppointmentID)
);

-- table Bills
DROP TABLE IF EXISTS Bills;

CREATE TABLE Bills (
    BillID INTEGER PRIMARY KEY,
    AppointmentID INTEGER NOT NULL,
    BillAmount REAL NOT NULL CHECK (BillAmount > 0),
    BillDate DATE NOT NULL,
    FOREIGN KEY (AppointmentID) REFERENCES Appointments(AppointmentID)
);

-- table InsuranceProviders
DROP TABLE IF EXISTS InsuranceProviders;

CREATE TABLE InsuranceProviders (
    ProviderID INTEGER PRIMARY KEY,
    ProviderName TEXT UNIQUE NOT NULL
);

```

```

-- table InsuranceClaims
DROP TABLE IF EXISTS InsuranceClaims;

CREATE TABLE InsuranceClaims (
    ClaimID INTEGER PRIMARY KEY,
    BillID INTEGER NOT NULL,
    ProviderID INTEGER,
    InsuranceCoveredAmount REAL NOT NULL CHECK (InsuranceCoveredAmount >= 0),
    ClaimStatus TEXT CHECK (ClaimStatus IN ('Approved', 'Rejected', 'Pending')),
    FOREIGN KEY (BillID) REFERENCES Bills(BillID),
    FOREIGN KEY (ProviderID) REFERENCES InsuranceProviders(ProviderID)
);

-- insert Specializations

INSERT INTO Specializations VALUES (1,'Cardiology');
INSERT INTO Specializations VALUES (2,'Dermatology');
INSERT INTO Specializations VALUES (3,'Orthopedics');
INSERT INTO Specializations VALUES (4,'Neurology');
INSERT INTO Specializations VALUES (5,'General Medicine');

-- insert Patients
INSERT INTO Patients VALUES (1,'Divya','1998-05-10','Female','9100000001','divya@gmail.com','Hyderabad');
INSERT INTO Patients VALUES (2,'Monishka','1997-08-12','Female','9100000002','monishka@gmail.com','Chennai');
INSERT INTO Patients VALUES (3,'Chiru','1995-02-20','Male','9100000003','chiru@gmail.com','Bangalore');
INSERT INTO Patients VALUES (4,'Anil','1993-01-11','Male','9100000004','anil@gmail.com','Delhi');
INSERT INTO Patients VALUES (5,'Parnika','1999-06-25','Female','9100000005','parnika@gmail.com','Hyderabad');

-- insert Doctors
INSERT INTO Doctors VALUES (1,'Dr. Rao','9000011111',1);
INSERT INTO Doctors VALUES (2,'Dr. Kumar','9000022222',2);
INSERT INTO Doctors VALUES (3,'Dr. Reddy','9000033333',3);
INSERT INTO Doctors VALUES (4,'Dr. Meena','9000044444',4);
INSERT INTO Doctors VALUES (5,'Dr. Siva','9000055555',5);

-- insert Appointments
INSERT INTO Appointments VALUES (1,1,1,'2024-01-05','Completed');
INSERT INTO Appointments VALUES (2,2,2,'2024-01-07','Completed');
INSERT INTO Appointments VALUES (3,3,3,'2024-01-10','Completed');
INSERT INTO Appointments VALUES (4,4,4,'2022-02-15','Completed');
INSERT INTO Appointments VALUES (5,5,5,'2024-01-20','Completed');

-- insert Treatments
INSERT INTO Treatments VALUES (1,1,'Heart Checkup',5000);
INSERT INTO Treatments VALUES (2,2,'Skin Treatment',3000);
INSERT INTO Treatments VALUES (3,3,'Bone X-Ray',2500);
INSERT INTO Treatments VALUES (4,4,'Neuro Scan',8000);
INSERT INTO Treatments VALUES (5,5,'General Consultation',1500);

-- insert Bills
INSERT INTO Bills VALUES (1,1,5000,'2024-01-05');
INSERT INTO Bills VALUES (2,2,3000,'2024-01-07');
INSERT INTO Bills VALUES (3,3,2500,'2024-01-10');
INSERT INTO Bills VALUES (4,4,8000,'2022-02-15');
INSERT INTO Bills VALUES (5,5,1500,'2024-01-20');

-- insert InsuranceProviders

```

```
INSERT INTO InsuranceProviders VALUES (1,'Health Secure');
INSERT INTO InsuranceProviders VALUES (2,'Life Care Insurance');
```

```
-- insert InsuranceClaims
```

```
INSERT INTO InsuranceClaims VALUES (1,1,1,2000,'Approved');
INSERT INTO InsuranceClaims VALUES (2,2,2,1000,'Approved');
INSERT INTO InsuranceClaims VALUES (3,3,NULL,0,'Rejected');
```

```
-- A Patient Billing Report
```

```
SELECT
    p.FullName AS PatientName,
    d.FullName AS DoctorName,
    s.SpecializationName,
    t.TreatmentName,
    a.AppointmentDate,
    b.BillAmount,
    IFNULL(ic.InsuranceCoveredAmount,0) AS InsuranceCoveredAmount,
    (b.BillAmount - IFNULL(ic.InsuranceCoveredAmount,0)) AS FinalPayableAmount
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Patients p ON a.PatientID = p.PatientID
JOIN Doctors d ON a.DoctorID = d.DoctorID
JOIN Specializations s ON d.SpecializationID = s.SpecializationID
JOIN Treatments t ON a.AppointmentID = t.AppointmentID
LEFT JOIN InsuranceClaims ic ON b.BillID = ic.BillID;
```

```
-- B Top 3 Revenue Generating Doctors
```

```
SELECT
    d.FullName,
    SUM(b.BillAmount) AS TotalRevenue
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Doctors d ON a.DoctorID = d.DoctorID
GROUP BY d.FullName
ORDER BY TotalRevenue DESC
LIMIT 3;
```

```
-- C Identify Patients with Pending Payments
```

```
SELECT
    p.FullName,
    SUM(b.BillAmount - IFNULL(ic.InsuranceCoveredAmount,0)) AS FinalPayableAmount,
    CASE
        WHEN SUM(b.BillAmount - IFNULL(ic.InsuranceCoveredAmount,0)) > 0 THEN 'Pending'
        ELSE 'Paid'
    END AS PaymentStatus
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Patients p ON a.PatientID = p.PatientID
LEFT JOIN InsuranceClaims ic ON b.BillID = ic.BillID
GROUP BY p.FullName;
```

```
-- D Specializations Generating Revenue Above Average
```

```
SELECT
    s.SpecializationName,
    SUM(b.BillAmount) AS TotalRevenue
FROM Bills b
JOIN Appointments a ON b.AppointmentID = a.AppointmentID
JOIN Doctors d ON a.DoctorID = d.DoctorID
```

```

JOIN Specializations s ON d.SpecializationID = s.SpecializationID
GROUP BY s.SpecializationName
HAVING SUM(b.BillAmount) >
    (SELECT AVG(BillAmount) FROM Bills);

```

```

-- E Delete Inactive Patients (no appointments in 3 years)
DELETE FROM Patients
WHERE NOT EXISTS (
    SELECT 1 FROM Appointments a
    WHERE a.PatientID = Patients.PatientID
    AND a.AppointmentDate >= DATE('now','-3 years')
);

```

```

-- F Give 5% bonus to doctors with revenue > 10 lakhs
UPDATE Doctors
SET Salary = Salary * 1.05
WHERE DoctorID IN (
    SELECT a.DoctorID
    FROM Bills b
    JOIN Appointments a
        ON b.AppointmentID = a.AppointmentID
    GROUP BY a.DoctorID
    HAVING SUM(b.BillAmount) > 1000000
);

```

Project2:

```

DROP TABLE IF EXISTS Reviews;
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Payments;

```

```

DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Categories;
DROP TABLE IF EXISTS Users;

```

```

--Users Table(buyer/seller)
CREATE TABLE Users (
    UserID INTEGER PRIMARY KEY,
    FullName TEXT NOT NULL,
    Email TEXT NOT NULL UNIQUE,
    Password TEXT NOT NULL,
    UserType TEXT CHECK (UserType IN ('Buyer','Seller')) NOT NULL
);

```

```

--Categories table
CREATE TABLE Categories (
    CategoryID INTEGER PRIMARY KEY,
    CategoryName TEXT NOT NULL UNIQUE
);

```

```

--Product table
CREATE TABLE Products (
    ProductID INTEGER PRIMARY KEY,
    ProductName TEXT NOT NULL,
    Price REAL NOT NULL CHECK (Price > 0),
    Stock INTEGER NOT NULL CHECK (Stock >= 0),
    SellerID INTEGER NOT NULL,
    CategoryID INTEGER NOT NULL,

```

```

FOREIGN KEY (SellerID) REFERENCES Users(UserID),
FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)
);
DROP TABLE IF EXISTS Orders;
--orders Table
CREATE TABLE Orders (
    OrderID INTEGER PRIMARY KEY,
    BuyerID INTEGER NOT NULL,
    OrderDate TEXT DEFAULT CURRENT_DATE,
    Status TEXT CHECK (Status IN ('Pending','Delivered','Cancelled')) DEFAULT 'Pending',
    FOREIGN KEY (BuyerID) REFERENCES Users(UserID)
);
--ON DELETE CASCADE for OrderDetails it removes orderdetails for deleted order automatically
CREATE TABLE OrderDetails (
    OrderDetailID INTEGER PRIMARY KEY,
    OrderID INTEGER NOT NULL,
    ProductID INTEGER NOT NULL,
    Quantity INTEGER NOT NULL CHECK (Quantity > 0),
    UnitPrice REAL NOT NULL CHECK (UnitPrice > 0),

    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
        ON DELETE CASCADE,

    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);
--Payment Table
CREATE TABLE Payments (
    PaymentID INTEGER PRIMARY KEY,
    OrderID INTEGER UNIQUE,
    Amount REAL NOT NULL CHECK (Amount > 0),
    PaymentDate TEXT DEFAULT CURRENT_DATE,

    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
);
--Review Table
CREATE TABLE Reviews (
    ReviewID INTEGER PRIMARY KEY,
    ProductID INTEGER NOT NULL,
    BuyerID INTEGER NOT NULL,
    Rating INTEGER NOT NULL CHECK (Rating BETWEEN 1 AND 5),
    Comment TEXT,

    FOREIGN KEY (ProductID) REFERENCES Products(ProductID),
    FOREIGN KEY (BuyerID) REFERENCES Users(UserID)
);
--insertion(buyer/seller)
INSERT INTO Users VALUES (1, 'Aman Khanna', 'aman.khanna@gmail.com', 'Aman@123', 'Buyer');
INSERT INTO Users VALUES (2, 'Ritika Malhotra', 'ritika.malhotra@gmail.com', 'Ritika@456', 'Seller');
INSERT INTO Users VALUES (3, 'Saurabh Jain', 'saurabh.jain@yahoo.com', 'Saurabh@789', 'Buyer');
INSERT INTO Users VALUES (4, 'Neha Bansal', 'neha.bansal@gmail.com', 'Neha@321', 'Seller');
INSERT INTO Users VALUES (5, 'Vikram Singh', 'vikram.singh@gmail.com', 'Vikram@654', 'Buyer');

--categories
INSERT INTO Categories VALUES (1, 'Electronics');
INSERT INTO Categories VALUES (2, 'Clothing');
INSERT INTO Categories VALUES (3, 'Books');
--Products

```

```

INSERT INTO Products VALUES (101, 'Wireless Keyboard', 1499.00, 25, 2, 1);
INSERT INTO Products VALUES (102, 'Noise Cancelling Headphones', 5999.00, 15, 2, 1);
INSERT INTO Products VALUES (103, 'Men Cotton Shirt', 899.00, 40, 4, 2);
INSERT INTO Products VALUES (104, 'Data Structures Book', 699.00, 30, 4, 3);
INSERT INTO Products VALUES (105, 'Bluetooth Speaker', 2499.00, 20, 2, 1);
---orders
INSERT INTO Orders VALUES (2001, 1, '2026-02-20', 'Delivered');
INSERT INTO Orders VALUES (2002, 3, '2026-02-21', 'Delivered');
INSERT INTO Orders VALUES (2003, 5, '2026-02-22', 'Pending');
INSERT INTO Orders VALUES (2004, 2, '2026-02-23', 'Delivered');
--orderdetails
INSERT INTO OrderDetails VALUES (1, 2001, 101, 2, 1499.00);
INSERT INTO OrderDetails VALUES (2, 2001, 104, 1, 699.00);
INSERT INTO OrderDetails VALUES (3, 2002, 103, 3, 899.00);
INSERT INTO OrderDetails VALUES (4, 2003, 102, 1, 5999.00);
INSERT INTO OrderDetails VALUES (5, 2004, 105, 2, 2499.00);
--payment
INSERT INTO Payments VALUES (1, 2001, 3697.00, '2026-02-20');
INSERT INTO Payments VALUES (2, 2002, 2697.00, '2026-02-21');
INSERT INTO Payments VALUES (3, 2003, 5999.00, '2026-02-22');
INSERT INTO Payments VALUES (4, 2004, 4998.00, '2026-02-23');
--Reviews

INSERT INTO Reviews VALUES (5, 105, 1, 4, 'Good bass and battery backup. ');
INSERT INTO Reviews VALUES (1, 101, 1, 5, 'Very smooth typing experience. ');
INSERT INTO Reviews VALUES (2, 104, 1, 4, 'Helpful for exam preparation. ');
INSERT INTO Reviews VALUES (3, 103, 3, 3, 'Good quality but size runs small. ');
INSERT INTO Reviews VALUES (4, 102, 5, 5, 'Amazing sound clarity! ');

```

--Seller Performance Dashboard

```

SELECT
    U.FullName AS SellerName,

    COUNT(DISTINCT P.ProductID) AS TotalProducts,

    COUNT(DISTINCT O.OrderID) AS TotalOrders,

    IFNULL(SUM(OD.Quantity * OD.UnitPrice), 0) AS TotalRevenue,

    ROUND(IFNULL(AVG(R.Rating), 0), 2) AS AverageRating,

    CASE
        WHEN IFNULL(SUM(OD.Quantity * OD.UnitPrice), 0) > 500000
            THEN 'Platinum'
        WHEN IFNULL(SUM(OD.Quantity * OD.UnitPrice), 0) >= 200000
            THEN 'Gold'
        ELSE 'Silver'
    END AS PerformanceLevel

```

FROM Users U

```

LEFT JOIN Products P
    ON U.UserID = P.SellerID

```

```

LEFT JOIN OrderDetails OD
    ON P.ProductID = OD.ProductID

```

```

LEFT JOIN Orders O

```

```

ON OD.OrderID = O.OrderID
AND O.Status = 'Delivered'

LEFT JOIN Reviews R
  ON P.ProductID = R.ProductID

WHERE U.UserType = 'Seller'

GROUP BY U.UserID

ORDER BY TotalRevenue DESC;
--Detect Suspicious Buyers
SELECT
  U.UserID AS BuyerID,
  U.FullName AS BuyerName
FROM Users U
WHERE U.UserType = 'Buyer'
AND (

  -- Condition 1: More than 5 orders in one day
  U.UserID IN (
    SELECT O.BuyerID
    FROM Orders O
    GROUP BY O.BuyerID, O.OrderDate
    HAVING COUNT(O.OrderID) > 5
  )

  OR

  -- Condition 2: Cancelled orders more than 50% of total orders
  U.UserID IN (
    SELECT O.BuyerID
    FROM Orders O
    GROUP BY O.BuyerID
    HAVING
      SUM(CASE WHEN O.Status = 'Cancelled' THEN 1 ELSE 0 END)
      > 0.5 * COUNT(O.OrderID)
  )

);
--Find Most Profitable Category
SELECT
  C.CategoryName,
  SUM(OD.Quantity * OD.UnitPrice) AS TotalRevenue
FROM Categories C

JOIN Products P
  ON C.CategoryID = P.CategoryID

JOIN OrderDetails OD
  ON P.ProductID = OD.ProductID

JOIN Orders O
  ON OD.OrderID = O.OrderID
  AND O.Status = 'Delivered' -- only delivered orders

GROUP BY C.CategoryID

```

ORDER BY TotalRevenue DESC

LIMIT 1;

--Delete Inactive Sellers

DELETE FROM Users

WHERE UserType = 'Seller'

AND (

-- No Products

UserID NOT IN (

SELECT SellerID

FROM Products

)

OR

-- No orders in last 1 year

UserID NOT IN (

SELECT DISTINCT P.SellerID

FROM Products P

JOIN OrderDetails OD ON P.ProductID = OD.ProductID

JOIN Orders O ON OD.OrderID = O.OrderID

WHERE O.OrderDate >= DATE('now', '-1 year')

)

);

-----Apply 10% Discount to Slow-Moving Products

UPDATE Products

SET Price = Price * 0.90 -- 10% discount

WHERE ProductID IN (

SELECT P.ProductID

FROM Products P

WHERE

-- Total quantity sold of that product

IFNULL((

SELECT SUM(OD.Quantity)

FROM OrderDetails OD

WHERE OD.ProductID = P.ProductID

), 0)

<

-- Average quantity sold of that category

(

SELECT AVG(

IFNULL((

SELECT SUM(OD2.Quantity)

FROM OrderDetails OD2

WHERE OD2.ProductID = P2.ProductID

), 0)

)

FROM Products P2

WHERE P2.CategoryID = P.CategoryID

)

);

--Find Products Never Reviewed

```

SELECT
    P.ProductID,
    P.ProductName
FROM Products P
WHERE NOT EXISTS (

    SELECT 1
    FROM Reviews R
    WHERE R.ProductID = P.ProductID

);
*****

Project3: --Cities
DROP TABLE IF EXISTS Cities;
CREATE TABLE Cities (
    CityID INT PRIMARY KEY,
    CityName VARCHAR(100) UNIQUE NOT NULL
);

--Drivers
DROP TABLE IF EXISTS Drivers;
CREATE TABLE Drivers (
    DriverID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(20) UNIQUE NOT NULL,
    CityID INT NOT NULL,
    JoinDate DATE NOT NULL,
    FOREIGN KEY (CityID) REFERENCES Cities(CityID)
);

--Riders
DROP TABLE IF EXISTS Riders;
CREATE TABLE Riders (
    RiderID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Phone VARCHAR(20) UNIQUE NOT NULL
);

--Vehicles
DROP TABLE IF EXISTS Vehicles;
CREATE TABLE Vehicles (
    VehicleID INT PRIMARY KEY,
    DriverID INT UNIQUE NOT NULL,
    Type VARCHAR(50) NOT NULL,
    PlateNumber VARCHAR(20) UNIQUE NOT NULL,
    FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID)
);

--Trips
DROP TABLE IF EXISTS Trips;
CREATE TABLE Trips (
    TripID INT PRIMARY KEY,
    DriverID INT NOT NULL,
    RiderID INT NOT NULL,
    CityID INT NOT NULL,
    TripDate DATE NOT NULL,
    DistanceKM DECIMAL(6,2) CHECK (DistanceKM > 0),
    FareAmount DECIMAL(10,2) CHECK (FareAmount > 0),

```

```

FOREIGN KEY (DriverID) REFERENCES Drivers(DriverID),
FOREIGN KEY (RiderID) REFERENCES Riders(RiderID),
FOREIGN KEY (CityID) REFERENCES Cities(CityID)
);

--Payments
DROP TABLE IF EXISTS Payments;
CREATE TABLE Payments (
    PaymentID INT PRIMARY KEY,
    TripID INT UNIQUE NOT NULL REFERENCES Trips(TripID) ON DELETE CASCADE,
    PaymentMethod VARCHAR(50) NOT NULL,
    PaidAmount DECIMAL(10,2) NOT NULL
);

--DriverRatings
DROP TABLE IF EXISTS DriverRatings;
CREATE TABLE DriverRatings (
    RatingID INT PRIMARY KEY,
    TripID INT UNIQUE NOT NULL REFERENCES Trips(TripID) ON DELETE CASCADE,
    RatingScore INT CHECK (RatingScore BETWEEN 1 AND 5)
);

--Insert data
--Cities
INSERT INTO Cities VALUES (1,'Hyderabad');
INSERT INTO Cities VALUES (2,'Mumbai');
INSERT INTO Cities VALUES (3,'Bengaluru');
INSERT INTO Cities VALUES (4,'Chennai');
INSERT INTO Cities VALUES (5,'Delhi');

--Drivers
INSERT INTO Drivers VALUES (101,'Ravi Kumar','9876543210',1,'2022-05-12');
INSERT INTO Drivers VALUES (102,'Imran Shaikh','9823456712',2,'2021-08-19');
INSERT INTO Drivers VALUES (103,'Suresh Reddy','9849012345',1,'2023-02-10');
INSERT INTO Drivers VALUES (104,'Arjun Nair','9887766554',3,'2022-11-03');
INSERT INTO Drivers VALUES (105,'Manoj Sharma','9811122233',5,'2020-06-25');

--Riders
INSERT INTO Riders VALUES (201,'Priya Sharma','9000012345');
INSERT INTO Riders VALUES (202,'Rahul Verma','9000023456');
INSERT INTO Riders VALUES (203,'Sneha Iyer','9000034567');
INSERT INTO Riders VALUES (204,'Karan Mehta','9000045678');
INSERT INTO Riders VALUES (205,'Ananya Gupta','9000056789');

--Vehicles
INSERT INTO Vehicles VALUES (301,101,'Sedan','TS09AB1234');
INSERT INTO Vehicles VALUES (302,102,'SUV','MH02CD5678');
INSERT INTO Vehicles VALUES (303,103,'Hatchback','TS10EF4321');
INSERT INTO Vehicles VALUES (304,104,'Sedan','KA05GH8765');
INSERT INTO Vehicles VALUES (305,105,'SUV','DL01JK1122');

--Trips
INSERT INTO Trips VALUES (401,101,201,1,'2024-01-10',8.5,220.00);
INSERT INTO Trips VALUES (402,103,202,1,'2024-01-12',15.0,380.00);
INSERT INTO Trips VALUES (403,102,203,2,'2024-01-15',12.3,310.00);
INSERT INTO Trips VALUES (404,104,204,3,'2024-01-18',6.8,180.00);
INSERT INTO Trips VALUES (405,105,205,5,'2024-01-20',20.0,520.00);
INSERT INTO Trips VALUES (406,101,203,1,'2024-01-22',5.2,150.00);

```

--Payments

```
INSERT INTO Payments VALUES (501,401,'UPI',220.00);
INSERT INTO Payments VALUES (502,402,'Card',380.00);
INSERT INTO Payments VALUES (503,403,'Cash',310.00);
INSERT INTO Payments VALUES (504,404,'UPI',180.00);
INSERT INTO Payments VALUES (505,405,'Card',520.00);
INSERT INTO Payments VALUES (506,406,'UPI',150.00);
```

--DriverRatings

```
INSERT INTO DriverRatings VALUES (601,401,5);
INSERT INTO DriverRatings VALUES (602,402,4);
INSERT INTO DriverRatings VALUES (603,403,5);
INSERT INTO DriverRatings VALUES (604,404,3);
INSERT INTO DriverRatings VALUES (605,405,4);
INSERT INTO DriverRatings VALUES (606,406,5);
```

--Analysis

--

SELECT

```
    d.Name AS DriverName,
    c.CityName AS City,
    ROUND(AVG(r.RatingScore),2) AS AvgRating,
    COUNT(t.TripID) AS TotalTrips,
```

CASE

-- Underperforming Condition

WHEN

AVG(r.RatingScore) <

(

SELECT AVG(r2.RatingScore)

FROM Trips t2

JOIN DriverRatings r2 ON t2.TripID = r2.TripID

WHERE t2.CityID = d.CityID

)

AND

COUNT(t.TripID) <

(

SELECT COUNT(t3.TripID) * 1.0 /

COUNT(DISTINCT t3.DriverID)

FROM Trips t3

WHERE t3.CityID = d.CityID

)

THEN 'Underperforming'

-- Excellent Condition

WHEN

AVG(r.RatingScore) >=

(

SELECT AVG(r2.RatingScore)

FROM Trips t2

JOIN DriverRatings r2 ON t2.TripID = r2.TripID

WHERE t2.CityID = d.CityID

)

AND

COUNT(t.TripID) >=

(

SELECT COUNT(t3.TripID) * 1.0 /

COUNT(DISTINCT t3.DriverID)

```

        FROM Trips t3
        WHERE t3.CityID = d.CityID
    )
    THEN 'Excellent'

    ELSE 'Stable'

END AS Status

FROM Drivers d
JOIN Cities c ON d.CityID = c.CityID
LEFT JOIN Trips t ON d.DriverID = t.DriverID
LEFT JOIN DriverRatings r ON t.TripID = r.TripID

GROUP BY d.DriverID, d.Name, c.CityName, d.CityID;
-- Revenue Consistency Analysis
SELECT
    d.Name AS DriverName,
    COUNT(t.TripID) AS TotalTrips,
    ROUND(AVG(t.FareAmount),2) AS AvgFare,
    MAX(t.FareAmount) AS HighestFare
FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
GROUP BY d.DriverID, d.Name
HAVING
    COUNT(t.TripID) >= 5
    AND
    MAX(t.FareAmount) > 3 * AVG(t.FareAmount);
--- City Revenue Imbalance

SELECT
    c.CityName,
    SUM(t.FareAmount) AS CityTotalRevenue
FROM Cities c
JOIN Trips t ON c.CityID = t.CityID
GROUP BY c.CityID, c.CityName
HAVING
(
    -- Revenue generated by Top 20% drivers
    SELECT SUM(driver_rev.TotalRevenue)
    FROM
    (
        SELECT d.DriverID, SUM(t2.FareAmount) AS TotalRevenue
        FROM Drivers d
        JOIN Trips t2 ON d.DriverID = t2.DriverID
        WHERE d.CityID = c.CityID
        GROUP BY d.DriverID
        HAVING
        (
            -- Rank simulation:
            SELECT COUNT(*)
            FROM Drivers d2
            JOIN Trips t3 ON d2.DriverID = t3.DriverID
            WHERE d2.CityID = c.CityID
            GROUP BY d2.DriverID
            HAVING SUM(t3.FareAmount) > SUM(t2.FareAmount)
        ) <
    ) <

```

```

        SELECT COUNT(*) * 0.2
        FROM Drivers d3
        WHERE d3.CityID = c.CityID
    )
) AS driver_rev
)
>
0.6 * SUM(t.FareAmount);
---Detect Riders Using Multiple Cities in Same Day
SELECT
    RiderID,
    TripDate,
    COUNT(DISTINCT CityID) AS CitiesVisited
FROM Trips
GROUP BY RiderID, TripDate
HAVING COUNT(DISTINCT CityID) > 1;

---Driver Loyalty Bonus Update
UPDATE Trips
SET FareAmount = FareAmount * 1.05
WHERE t.DriverID IN
(
    SELECT d.DriverID
    FROM Drivers d
    WHERE
        -- Condition 1: More trips than city average
        (
            SELECT COUNT(*)
            FROM Trips t2
            WHERE t2.DriverID = d.DriverID
        )
        >
        (
            SELECT COUNT(*) * 1.0 /
                COUNT(DISTINCT DriverID)
            FROM Trips t3
            WHERE t3.CityID = d.CityID
        )
    AND

        -- Condition 2: Driver rating > overall average
        (
            SELECT AVG(r.RatingScore)
            FROM Trips t4
            JOIN DriverRatings r
            ON t4.TripID = r.TripID
            WHERE t4.DriverID = d.DriverID
        )
        >
        (
            SELECT AVG(RatingScore)
            FROM DriverRatings
        )
);
--Delete Inactive Drivers
DELETE FROM Drivers d
WHERE

```

```

-- Condition 1: Driver has zero trips
NOT EXISTS
(
    SELECT 1
    FROM Trips t
    WHERE t.DriverID = d.DriverID
)

OR

-- Condition 2: No trips in last 2 years
NOT EXISTS
(
    SELECT 1
    FROM Trips t2
    WHERE t2.DriverID = d.DriverID
    AND t2.TripDate >= DATE('now', '-2 years')
);
--Fraud Detection Logic
SELECT
    t.TripID,
    d.Name AS DriverName,
    c.CityName AS City,

CASE
    WHEN
        t.DistanceKM < 1
        AND
        t.FareAmount >
        (
            SELECT AVG(t2.FareAmount)
            FROM Trips t2
            WHERE t2.CityID = t.CityID
        )
    THEN 'Suspicious'

    WHEN
        p.PaymentMethod = 'Cash'
        AND t.FareAmount > 5000
    THEN 'High Risk'

    ELSE 'Normal'

END AS RiskLevel

FROM Trips t
JOIN Drivers d ON t.DriverID = d.DriverID
JOIN Cities c ON t.CityID = c.CityID
JOIN Payments p ON t.TripID = p.TripID;
--Multi-Level Business Metric
SELECT
    c.CityName,
    COUNT(t.TripID) AS TotalTrips,
    SUM(t.FareAmount) / SUM(t.DistanceKM) AS CityFarePerKM
FROM Cities c
JOIN Trips t ON c.CityID = t.CityID
GROUP BY c.CityID, c.CityName
HAVING

```

```

-- Condition 1: City Fare per KM > Platform Fare per KM
(SUM(t.FareAmount) / SUM(t.DistanceKM))
>
(
    SELECT SUM(FareAmount) / SUM(DistanceKM)
    FROM Trips
)

AND

-- Condition 2: City Trips > Average Trips per City
COUNT(t.TripID)
>
(
    SELECT COUNT(*) * 1.0 / COUNT(DISTINCT CityID)
    FROM Trips
);

--Set Operator Challenge
--A:Drivers with Average Rating ≥ 4
SELECT d.DriverID, d.Name
FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
JOIN DriverRatings r ON t.TripID = r.TripID
GROUP BY d.DriverID, d.Name
HAVING AVG(r.RatingScore) >= 4

--B:Drivers with Revenue Above City Average Revenue
SELECT d.DriverID, d.Name
FROM Drivers d
JOIN Trips t ON d.DriverID = t.DriverID
GROUP BY d.DriverID, d.Name, d.CityID
HAVING
    SUM(t.FareAmount) >
    (
        SELECT SUM(t2.FareAmount) * 1.0 /
            COUNT(DISTINCT t2.DriverID)
        FROM Trips t2
        WHERE t2.CityID = d.CityID
    )

--Intersect satisfy both
(
    SELECT d.DriverID, d.Name
    FROM Drivers d
    JOIN Trips t ON d.DriverID = t.DriverID
    JOIN DriverRatings r ON t.TripID = r.TripID
    GROUP BY d.DriverID, d.Name
    HAVING AVG(r.RatingScore) >= 4
)
INTERSECT
(
    SELECT d.DriverID, d.Name
    FROM Drivers d
    JOIN Trips t ON d.DriverID = t.DriverID
    GROUP BY d.DriverID, d.Name, d.CityID
    HAVING
        SUM(t.FareAmount) >
        (

```

```

        SELECT SUM(t2.FareAmount) * 1.0 /
              COUNT(DISTINCT t2.DriverID)
        FROM Trips t2
        WHERE t2.CityID = d.CityID
    )
);
--Expect High Revenue but Low Rating
(
    SELECT d.DriverID, d.Name
    FROM Drivers d
    JOIN Trips t ON d.DriverID = t.DriverID
    GROUP BY d.DriverID, d.Name, d.CityID
    HAVING
        SUM(t.FareAmount) >
        (
            SELECT SUM(t2.FareAmount) * 1.0 /
                  COUNT(DISTINCT t2.DriverID)
            FROM Trips t2
            WHERE t2.CityID = d.CityID
        )
)
EXCEPT
(
    SELECT d.DriverID, d.Name
    FROM Drivers d
    JOIN Trips t ON d.DriverID = t.DriverID
    JOIN DriverRatings r ON t.TripID = r.TripID
    GROUP BY d.DriverID, d.Name
    HAVING AVG(r.RatingScore) >= 4
);
--Union
(
    SELECT d.DriverID, d.Name
    FROM Drivers d
    JOIN Trips t ON d.DriverID = t.DriverID
    JOIN DriverRatings r ON t.TripID = r.TripID
    GROUP BY d.DriverID, d.Name
    HAVING AVG(r.RatingScore) >= 4
)
UNION
(
    SELECT d.DriverID, d.Name
    FROM Drivers d
    JOIN Trips t ON d.DriverID = t.DriverID
    GROUP BY d.DriverID, d.Name, d.CityID
    HAVING
        SUM(t.FareAmount) >
        (
            SELECT SUM(t2.FareAmount) * 1.0 /
                  COUNT(DISTINCT t2.DriverID)
            FROM Trips t2
            WHERE t2.CityID = d.CityID
        )
);
--Advanced Integrity Validation Query
SELECT
    t.TripID,
    d.Name AS DriverName,

```

```
c1.CityName AS DriverCity,  
c2.CityName AS TripCity,  
t.FareAmount,  
p.PaidAmount,
```

```
CASE  
  WHEN p.PaidAmount <> t.FareAmount  
    AND d.CityID <> t.CityID  
  THEN 'Payment + City Mismatch'  
  
  WHEN p.PaidAmount <> t.FareAmount  
  THEN 'Payment Mismatch'  
  
  WHEN d.CityID <> t.CityID  
  THEN 'City Mismatch'  
  
  ELSE 'OK'  
END AS IssueType
```

```
FROM Trips t  
JOIN Drivers d ON t.DriverID = d.DriverID  
JOIN Cities c1 ON d.CityID = c1.CityID  
JOIN Cities c2 ON t.CityID = c2.CityID  
LEFT JOIN Payments p ON t.TripID = p.TripID
```

```
WHERE  
  p.PaidAmount <> t.FareAmount  
  OR  
  d.CityID <> t.CityID;
```

Html,css,js Project Date:26-02-2026

Project1:

1)Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Smart College Event Registration Portal</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Smart College Event Portal</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="events.html">Events</a>
      <a href="register.html">Register</a>
    </nav>
  </header>
  <!--main content-->
  <section class="home-main">
    <h2>Annual Tech Fest 2026</h2>
    <!-- used article for event descript ,independent data-->
    <article>
      <p>
        Welcome to our Annual Tech Fest 2026.
        This event includes coding competitions,
        project exhibitions, and cultural programs.
      </p>
    </article>
    <section class="card-section">
      <!--used oredrdlist for event schedule-->
      <div class="card">
        <h3>Event Schedule</h3>
        <ol>
          <li>10:00 AM - Inauguration Ceremony</li>
          <li>11:00 AM - Coding Competition</li>
          <li>02:00 PM - Project Exhibition</li>
          <li>04:00 PM - Prize Distribution</li>
        </ol>
      </div>
      <!--used unordered list for rules for event -->
      <div class="card">
        <h3>Rules & Guidelines</h3>
        <ul>
          <li>All participants must carry college ID</li>
          <li>Registration is mandatory.</li>
          <li>Maintain discipline during events.</li>
        </ul>
      </div>
    </section>
    <!--to get location-->
    <button type="button" onclick="getLocation()">Get Location</button>
    <p id="location"></p>
  </section>
  <footer>
```

```

    <p>© 2026 College Annual Event</p>
</footer>
    <script src="script.js"></script>
</body>
</html>
2)Events.html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Smart College Event Registration Portal</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header>
        <h1>Smart College Event Portal</h1>
        <nav>
            <a href="index.html">Home</a>
            <a href="events.html">Events</a>
            <a href="register.html">Register</a>
        </nav>
    </header>
    <section class="event-section">
        <h2>Upcoming Events</h2>
        <table class="event-table">
            <tr>
                <th>Event</th>
                <th>Date</th>
                <th>Venue</th>
                <th>Seats left</th>
            </tr>
            <tr>
                <td>Annual Tech Fest</td>
                <td>10 March 2026</td>
                <td>Main Auditorium</td>
                <td>
                    <!--meter to measure seats in the range-->
                    <meter value="40" min="0" max="100"></meter>
                    40/100
                </td>
            </tr>
            <tr>
                <td>Coding Challenge</td>
                <td>15 March 2026</td>
                <td>Lab 1</td>
                <td>
                    <meter value="15" min="0" max="50"></meter>
                    15/50
                </td>
            </tr>
        </table>
        <ul id="eventList"></ul>
    </section>
    <footer>
        <p>© 2026 College Annual Event</p>

```

```

</footer>
<script src="script.js"></script>

</body>
</html>
3)Register.html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Smart College Event Registration Portal</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Smart College Event Portal</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="events.html">Events</a>
      <a href="register.html">Register</a>
    </nav>
  </header>

  <!--main content-->
  <section class="register-form">
    <h2>Form</h2>
    <form class="event-form" >
      <label>Name:</label>
      <input type="text" id="name" onchange="nameChanged()">
      <label>Email:</label>
      <input type="email" id="email">
      <label>Phone No:</label>
      <input type="number" id="phone">
      <label>Gender</label>

      <label><input type="radio" name="gender" value="Male">Male</label>
      <label><input type="radio" name="gender" value="Female">Female</label>
      <label>Date:</label>
      <input type="date" id="date">
      <label>Select Events:</label>
      <input type="checkbox" name="event" value="techfest">Tech Fest<br>
      <input type="checkbox" name="event" value="coding"> Coding Challenge <br>
      <input type="checkbox" name="event" value="cultural"> Cultural Program <br>
      <label>Select Department</label>
      <input type="text" name="Department" list="dept-list">
      <datalist id="dept-list">
        <option value="Computer Science">
        <option value="Information Technology">
        <option value="Mechanical Engineering">
        <option value="Civil Engineering">
        <option value="Electronics">
      </datalist>
      <!--register button-->
      <button type="button" onclick="registerEvent()">Register</button>
    </form>
    <p id="message"></p><!--display msg from registerEvent() DOM Manipulation-->
  </section>

```

```
<footer>
  <p>© 2026 College Annual Event</p>
</footer>
<script src="script.js"></script>
</body>
</html>
*****Style.css
* {
  margin: 0;
  padding: 0;
}

header {
  background-color: antiquewhite;
  padding: 10px;
}

header h1 {
  text-align: center;
}

nav {
  display: flex;
  justify-content: space-between;
  padding: 10px;
  background-color: blueviolet;
  border: 1px solid yellow;
  border-radius: 10px;
}

nav a {
  flex: 1;
  text-align: center;
  text-decoration: none;
  background-color: aqua;
  margin: 5px;
  padding: 10px;
  border: 1px solid yellow;
  border-radius: 10px;
  color: black;
}

nav a:hover {
  background-color: rgb(30, 18, 193);
  color: white;
}

.home-main {
  padding: 20px;
  max-width: 1000px;
  margin: auto;
}

.home-main h2 {
  text-align: center;
  margin-bottom: 15px;
}
```

```
.card-section {
  display: flex;
  justify-content: space-between;
  gap: 20px;
  margin-top: 20px;
}
```

```
.card {
  flex: 1;
  padding: 15px;
  border: 1px solid gray;
  border-radius: 8px;
  background-color: whitesmoke;
}
```

```
.card h3 {
  text-align: center;
  margin-bottom: 10px;
}
```

```
.card ol,
.card ul {
  margin-left: 20px;
}
@media (max-width: 768px) {
  .card-section {
    flex-direction: column;
  }
}
```

```
footer {
  margin-top: 20px;
  padding: 10px;
  background-color: antiquewhite;
  text-align: center;
}
footer {
  margin-top: 20px;
  padding: 10px;
  background-color: antiquewhite;
}
```

```
footer p {
  text-align: center;
}
```

/** events**/

```
.event-table {
  width: 80%;
  margin: 30px auto;
  border-collapse: collapse;
}
```

```
.event-table th,
.event-table td {
  border: 1px solid black;
  padding: 12px;
  text-align: center;
}
```

```
.event-table th {
    background-color: purple;
    color: white;
}

.event-table tr:nth-child(even) {
    background-color: lightgray;
}

.event-table tr:hover {
    background-color: gray;
}

/** Register**/
.register-form {
    padding: 20px;
}

.register-form h2 {
    text-align: center;
    margin-bottom: 15px;
}

.event-form {
    width: 400px;
    margin: 20px auto;
    padding: 20px;
    border: 2px solid black;
    border-radius: 10px;
    background-color: lightgray;
}

.event-form label {
    display: block;
    margin-top: 10px;
    color: black;
}

.event-form input[type="text"],
.event-form input[type="email"],
.event-form input[type="number"],
.event-form input[type="date"] {
    width: 100%;
    padding: 8px;
    margin-top: 5px;
    border: 1px solid black;
    border-radius: 5px;
}

.event-form input[type="radio"],
.event-form input[type="checkbox"] {
    margin-right: 5px;
}

.event-form button {
    margin-top: 15px;
    padding: 10px;
    width: 100%;
}
```

```

    background-color: purple;
    color: white;
    border: 1px solid black;
    border-radius: 5px;
}

.event-form button:hover {
    background-color: green;
}
Script.js
let studentName = "Rahul";
let age = 20;
let isRegistered = false;

>//Loops for listing events
function displayEvents() {
    let events = ["Tech Fest", "Coding Challenge", "Cultural Program"];
    let list = document.getElementById("eventList");

    for (let i = 0; i < events.length; i++) {
        let li = document.createElement("li");
        li.textContent = events[i];
        list.appendChild(li);
    }
};

displayEvents();

//function to validate form and also IF_ELSE USEd
function validateForm() {

    let name = document.getElementById("name").value;
    let email = document.getElementById("email").value;
    let gender = document.querySelector("input[name='gender']:checked");

    if (name === "") {
        alert("Name is required");
        return false;
    }

    if (email === "") {
        alert("Email is required");
        return false;
    }

    if (!gender) {
        alert("Please select gender");
        return false;
    }

    alert("Form submitted successfully");
    return true;
}

// function for Event Registration stores data in localStorage
function registerEvent() {

    if (!validateForm()) {
        return;
    }

```

```

}

let name = document.getElementById("name").value;
let email = document.getElementById("email").value;
let phone = document.getElementById("phone").value;
let gender = document.querySelector("input[name='gender']:checked").value;

// Create object
let student = {
  name: name,
  email: email,
  phone: phone,
  gender: gender
};

// Convert object to string and storing object in browser by setitem()
localStorage.setItem("studentData", JSON.stringify(student));

document.getElementById("message").innerHTML =
  "Registration Successful, " + name; //DOM Manipulation show confirmation message
}

//Retrieve on reload
window.onload = function () {

  let data = localStorage.getItem("studentData");

  if (data) {

    let studentObj = JSON.parse(data);
    document.getElementById("message").innerHTML =
      "Welcome back, " + studentObj.name;
  }

};

//onclick it is active when user chnage value/moves outside the name field
function nameChanged() {
  let name = document.getElementById("name").value;

  document.getElementById("message").innerHTML =
    "You entered: " + name;
}

//geola
function getLocation() {

  if (navigator.geolocation) {

    navigator.geolocation.getCurrentPosition(showPosition, showError);

  } else {

    document.getElementById("location").innerHTML =
      "Geolocation is not supported by this browser.";
  }
}

function showPosition(position) {

```

```

let latitude = position.coords.latitude;
let longitude = position.coords.longitude;

document.getElementById("location").innerHTML =
    "Latitude: " + latitude + "<br>Longitude: " + longitude;
}

function showError(error) {

    if (error.code === 1) {
        document.getElementById("location").innerHTML =
            "Permission denied. Please allow location access.";
    }
    else if (error.code === 2) {
        document.getElementById("location").innerHTML =
            "Location unavailable.";
    }
    else if (error.code === 3) {
        document.getElementById("location").innerHTML =
            "Request timed out.";
    }
    else {
        document.getElementById("location").innerHTML =
            "An unknown error occurred.";
    }
}
}

```

Project2:

1)Index.html

```

<!DOCTYPE html>
<html>
<head>
    <title>Personal Finance Tracker</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

    <h1>Personal Finance Tracker</h1>

    <!-- form-->
    <h2>Add Transaction</h2>

    <form onsubmit="return addTransaction()">

        <label>Type:</label>
        <select id="type">
            <option value="income">Income</option>
            <option value="expense">Expense</option>
        </select>

        <label>Amount:</label>
        <input type="number" id="amount" required>

        <label>Date:</label>
        <input type="date" id="date" required>

        <label>Category:</label>
        <input type="text" id="category" list="catList" required>
    </form>

```

```

<datalist id="catList">
  <option value="Salary">
  <option value="Food">
  <option value="Transport">
  <option value="Shopping">
</datalist>

  <button type="submit">Add</button>
</form>

<hr>

<!-- table-->
<h2>Transactions</h2>

<table border="1">
  <tr>
    <th>Type</th>
    <th>Amount</th>
    <th>Date</th>
    <th>Category</th>
  </tr>
  <tbody id="tableBody"></tbody>
</table>

<h3>Total Balance: <span id="balance">0</span></h3>

<hr>

<!--progress-->
<h2>Monthly Savings Goal (10000)</h2>
<progress id="savingProgress" value="0" max="10000"></progress>

<h2>Expense Ratio</h2>
<meter id="expenseMeter" min="0" max="10000" value="0"></meter>

<hr>

<!--tips-->
<h2>Financial Tips</h2>
<ol>
  <li>Track daily expenses</li>
  <li>Save at least 20% income</li>
  <li>Avoid unnecessary spending</li>
</ol>

<script src="script.js"></script>

</body>
</html>
*****Style.css:
body {
  font-family: Arial;
  padding: 20px;
  background-color: lightgray;
}

```

```

form {
  padding: 15px;
  border: 1px solid black;
  margin-bottom: 20px;
}

input, select, button {
  display: block;
  margin: 10px 0;
  padding: 5px;
}

button:hover {
  background-color: lightblue;
}

table {
  width: 100%;
  border-collapse: collapse;
}

th {
  background-color: gray;
  color: white;
}

td, th {
  padding: 8px;
  text-align: center;
}
}
****Script.js
let transactions = [];

// Load from localStorage when page opens
window.onload = function () {
  let data = localStorage.getItem("financeData");

  if (data) {
    transactions = JSON.parse(data);
    updateTable();
    calculateTotal();
  }
};

// add transaction
function addTransaction() {

  let type = document.getElementById("type").value;
  let amount = Number(document.getElementById("amount").value);
  let date = document.getElementById("date").value;
  let category = document.getElementById("category").value;

  if (amount <= 0) {
    alert("Enter valid amount");
    return false;
  }

  let transaction = {

```

```

        type: type,
        amount: amount,
        date: date,
        category: category
    };

    transactions.push(transaction);

    localStorage.setItem("financeData", JSON.stringify(transactions));

    updateTable();
    calculateTotal();

    return false;
}

// Update table
function updateTable() {

    let table = document.getElementById("tableBody");
    table.innerHTML = "";

    transactions.forEach(function (t) {

        let row = document.createElement("tr");

        row.innerHTML =
            "<td>" + t.type + "</td>" +
            "<td>" + t.amount + "</td>" +
            "<td>" + t.date + "</td>" +
            "<td>" + t.category + "</td>";

        table.appendChild(row);
    });
}

// Calculate STotal
function calculateTotal() {

    let income = 0;
    let expense = 0;

    transactions.forEach(function (t) {

        if (t.type === "income") {
            income += t.amount;
        } else {
            expense += t.amount;
        }
    });

    let balance = income - expense;

    document.getElementById("balance").innerHTML = balance;

    document.getElementById("savingProgress").value = balance;

    document.getElementById("expenseMeter").value = expense;

```

```

}
*****

Project3:
1)Dashboard.html
<!DOCTYPE html>
<html>
<head>
  <title>Dashboard</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<nav>
  <a href="dashboard.html">Dashboard</a>
  <a href="courses.html">Courses</a>
  <a href="quiz.html">Quiz</a>
  <a href="profile.html">Profile</a>
</nav>

<p>Home > Dashboard</p>

<h1>Dashboard</h1>

<div class="dashboard-grid">
  <div class="card">Total Courses: 3</div>
  <div class="card">Completed: <span id="completedCount">0</span></div>
  <div class="card">Last Score: <span id="lastScore">0</span>%</div>
</div>

<script type="module" src="main.js"></script>
</body>
</html>
2)Course.html
<!DOCTYPE html>
<html>
<head>
  <title>Courses</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<nav>
  <a href="dashboard.html">Dashboard</a>
  <a href="courses.html">Courses</a>
  <a href="quiz.html">Quiz</a>
  <a href="profile.html">Profile</a>
</nav>

<p>Home > Courses</p>

<h1>Courses</h1>

<table>
  <tr>
    <th>ID</th>
    <th>Course</th>
    <th>Lessons</th>
  </tr>

```

```
<tbody id="courseTable"></tbody>
</table>
```

```
<h2>Lessons</h2>
<ol>
  <li>Introduction</li>
  <li>Basics</li>
  <li>Examples</li>
</ol>
```

```
<h2>Progress</h2>
<progress id="courseProgress" value="0" max="100"></progress>
```

```
<script type="module" src="main.js"></script>
</body>
```

```
</html>
```

3)Quiz.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Quiz</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
<nav>
  <a href="dashboard.html">Dashboard</a>
  <a href="courses.html">Courses</a>
  <a href="quiz.html">Quiz</a>
  <a href="profile.html">Profile</a>
</nav>
```

```
<p>Home > Quiz</p>
```

```
<h1>Quiz</h1>
```

```
<div id="quizContainer"></div>
```

```
<button onclick="submitQuiz()">Submit Quiz</button>
```

```
<div id="result"></div>
```

```
<script type="module" src="main.js"></script>
</body>
```

```
</html>
```

4)Profile.html:

```
<!DOCTYPE html>
<html>
<head>
  <title>Profile</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
<nav>
  <a href="dashboard.html">Dashboard</a>
  <a href="courses.html">Courses</a>
  <a href="quiz.html">Quiz</a>
```

```
<a href="profile.html">Profile</a>
</nav>
```

```
<p>Home > Profile</p>
```

```
<h1>User Profile</h1>
```

```
<p>Last Quiz Score: <span id="profileScore"></span>%</p>
```

```
<p>Completed Courses: <span id="profileCompleted"></span></p>
```

```
<script type="module" src="main.js"></script>
```

```
</body>
```

```
</html>
```

```
** style.css
```

```
body {
```

```
  font-family: Arial;
```

```
  margin: 0;
```

```
  padding: 20px;
```

```
}
```

```
nav {
```

```
  background: #333;
```

```
  padding: 10px;
```

```
}
```

```
nav a {
```

```
  color: white;
```

```
  margin-right: 15px;
```

```
  text-decoration: none;
```

```
}
```

```
.dashboard-grid {
```

```
  display: grid;
```

```
  grid-template-columns: repeat(3, 1fr);
```

```
  gap: 15px;
```

```
}
```

```
.card {
```

```
  border: 1px solid gray;
```

```
  padding: 15px;
```

```
}
```

```
.course-container {
```

```
  display: flex;
```

```
  gap: 15px;
```

```
  flex-wrap: wrap;
```

```
}
```

```
.course-card {
```

```
  border: 1px solid black;
```

```
  padding: 15px;
```

```
  width: 200px;
```

```
}
```

```
table {
```

```
  border-collapse: collapse;
```

```
  width: 100%;
```

```
}
```

```
table, th, td {
  border: 1px solid black;
  padding: 8px;
}
```

```
progress {
  width: 100%;
}
```

```
@media (max-width: 768px) {
  .dashboard-grid {
    grid-template-columns: 1fr;
  }
}
```

```
**data.js:
```

```
// Store courses as objects
```

```
export const courses = [
```

```
  { id: 1, title: "HTML Basics", lessons: 5 },
```

```
  { id: 2, title: "CSS Fundamentals", lessons: 6 },
```

```
  { id: 3, title: "JavaScript Basics", lessons: 8 }
```

```
];
```

```
// Store quiz questions in array
```

```
export const quizQuestions = [
```

```
{
```

```
  question: "What does HTML stand for?",
```

```
  options: ["Hyper Text Markup Language", "Home Tool Markup Language", "Hyperlinks Text Mark Language"],
```

```
  answer: 0
```

```
},
```

```
{
```

```
  question: "Which CSS property changes text color?",
```

```
  options: ["font-style", "color", "background"],
```

```
  answer: 1
```

```
},
```

```
{
```

```
  question: "Which keyword is used to declare variable in ES6?",
```

```
  options: ["var", "int", "let"],
```

```
  answer: 2
```

```
}
```

```
];
```

```
**main.js
```

```
import { courses, quizQuestions } from "./data.js";
```

```
// Load course table
```

```
const table = document.getElementById("courseTable");
```

```
if (table) {
```

```
  courses.forEach(course => {
```

```
    table.innerHTML += `<tr>
```

```
      <td>${course.id}</td>
```

```
      <td>${course.title}</td>
```

```
      <td>${course.lessons}</td>
```

```
    </tr>`;
```

```
  });
```

```
}
```

```
// Load dashboard data
```

```
if (document.getElementById("completedCount")) {
```

```

document.getElementById("completedCount").innerText =
  localStorage.getItem("completedCourses") || 0;

document.getElementById("lastScore").innerText =
  localStorage.getItem("quizScore") || 0;
}

// Load profile data
if (document.getElementById("profileScore")) {
  document.getElementById("profileScore").innerText =
    localStorage.getItem("quizScore") || 0;

  document.getElementById("profileCompleted").innerText =
    localStorage.getItem("completedCourses") || 0;
}

// Async load quiz using Promise
function loadQuiz() {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve(quizQuestions);
    }, 1000);
  });
}

// Generate quiz questions
async function generateQuiz() {
  const container = document.getElementById("quizContainer");
  if (!container) return;

  const questions = await loadQuiz();

  questions.forEach((q, index) => {
    container.innerHTML += `<p>${q.question}</p>`;
    q.options.forEach((opt, i) => {
      container.innerHTML += `
        <input type="radio" name="q${index}" value="${i}" onchange="selectAnswer()">
        ${opt}<br>`;
    });
  });
}

generateQuiz();

// Submit quiz
window.submitQuiz = function () {
  let score = 0;

  quizQuestions.forEach((q, index) => {
    let selected = document.querySelector(`input[name="q${index}"]:checked`);
    if (selected && Number(selected.value) === q.answer) {
      score++;
    }
  });

  let percentage = (score / quizQuestions.length) * 100;

  localStorage.setItem("quizScore", percentage);

```

```

let resultDiv = document.getElementById("result");

// if-else grading
if (percentage >= 60) {
  resultDiv.innerHTML = "Passed! Score: " + percentage + "%";
} else {
  resultDiv.innerHTML = "Failed! Score: " + percentage + "%";
}

// switch feedback
switch (true) {
  case percentage === 100:
    resultDiv.innerHTML += "<br>Excellent!";
    break;
  case percentage >= 60:
    resultDiv.innerHTML += "<br>Good Job!";
    break;
  default:
    resultDiv.innerHTML += "<br>Need Improvement!";
}
};
**test.jss:
// Grade calculation test
test("Calculate percentage correctly", () => {
  expect((3 / 3) * 100).toBe(100);
});

// Pass logic test
test("Pass if >= 60", () => {
  let percentage = 70;
  expect(percentage >= 60).toBe(true);
});

// Fail logic test
test("Fail if < 60", () => {
  let percentage = 40;
  expect(percentage >= 60).toBe(false);
});
*****

```

Project4:

1)Index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Geo Task Manager</title>
```

```
<link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<header>
```

```
<h1>Field Employee Task Manager</h1>
```

```
</header>
```

```
<section class="grid-container">
```

```
<!-- Task Form -->
```

```
<div class="form-box">
```

<h2>Add Task</h2>

<form id="taskForm">

<label>Task Title:</label>

<input type="text" id="title" required>

<label>Due Date:</label>

<input type="date" id="date" required>

<label>Priority:</label>

<input list="priorityList" id="priority" required>

<datalist id="priorityList">

<option value="Low">

<option value="Medium">

<option value="High">

</datalist>

<label>Estimated Hours:</label>

<input type="number" id="hours" min="1" required>

<label>

<input type="checkbox" id="completed">

Completed

</label>

<button type="submit">Add Task</button>

</form>

</div>

<!-- Task Table -->

<div class="table-box">

<h2>Task List</h2>

<table>

<thead>

<tr>

<th>ID</th>

<th>Title</th>

<th>Priority</th>

<th>Location</th>

<th>Status</th>

</tr>

</thead>

<tbody id="taskTable"></tbody>

</table>

<h3>Completion Progress</h3>

<progress id="taskProgress" value="0" max="100"></progress>

<h3>High Priority Load</h3>

<meter id="priorityMeter" min="0" max="100" value="0"></meter>

</div>

</section>

```

<script src="main.js"></script>
</body>
</html>
2)style.css
body {
  font-family: Arial;
  margin: 0;
  padding: 20px;
}

header {
  text-align: center;
}

.grid-container {
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 20px;
}

.form-box, .table-box {
  border: 1px solid gray;
  padding: 15px;
}

form {
  display: flex;
  flex-direction: column;
}

form input, form button {
  margin-bottom: 10px;
  padding: 8px;
}

table {
  width: 100%;
  border-collapse: collapse;
}

table, th, td {
  border: 1px solid black;
  padding: 8px;
}

/* Advanced selector */
table tbody tr:nth-child(even) {
  background-color: #f2f2f2;
}

/* Media Query */
@media (max-width: 768px) {
  .grid-container {
    grid-template-columns: 1fr;
  }
}
3.main.js:
let tasks = [];

```

```

// Load tasks from localStorage
window.onload = function () {
  let stored = localStorage.getItem("tasks");
  if (stored) {
    tasks = JSON.parse(stored);
    displayTasks();
  }
};

// Form submit
document.getElementById("taskForm").addEventListener("submit", async function (e) {
  e.preventDefault();

  let title = document.getElementById("title").value;
  let priority = document.getElementById("priority").value;
  let completed = document.getElementById("completed").checked;

  getLocationAndCreateTask(title, priority, completed);
});

// Get location
function getLocationAndCreateTask(title, priority, completed) {

  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(
      function (position) {
        let location = position.coords.latitude + ", " + position.coords.longitude;
        createTask(title, priority, location, completed);
      },
      function (error) {
        if (error.code === 1) {
          alert("Permission denied for location.");
        } else if (error.code === 2) {
          alert("Location unavailable.");
        } else if (error.code === 3) {
          alert("Location timeout.");
        } else {
          alert("Unknown location error.");
        }
      }
    );
  } else {
    alert("Geolocation not supported.");
  }
}

// Create task object
async function createTask(title, priority, location, completed) {

  let newTask = {
    id: Date.now(),
    title: title,
    priority: priority,
    location: location,
    completed: completed
  };

```

```

    try {
      await simulateServerSave(newTask);
      tasks.push(newTask);
      saveTasks();
      displayTasks();
    } catch (error) {
      alert("Error saving task to server.");
    }
  }
}

// Simulate server save
function simulateServerSave(task) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      let success = true;
      if (success) {
        resolve("Saved");
      } else {
        reject("Error");
      }
    }, 1000);
  });
}

// Save to localStorage
function saveTasks() {
  localStorage.setItem("tasks", JSON.stringify(tasks));
}

// Display tasks
function displayTasks() {

  let table = document.getElementById("taskTable");
  table.innerHTML = "";

  tasks.map(task => {
    table.innerHTML += `
      <tr>
        <td>${task.id}</td>
        <td>${task.title}</td>
        <td>${task.priority}</td>
        <td>${task.location}</td>
        <td>${task.completed ? "Done" : "Pending"}</td>
      </tr>
    `;
  });

  updateProgress();
}

// Update progress and meter
function updateProgress() {

  let completedTasks = tasks.filter(t => t.completed).length;
  let total = tasks.length;

  let percent = total === 0 ? 0 : (completedTasks / total) * 100;

```

```
document.getElementById("taskProgress").value = percent;

// High priority load using find + filter
let highPriority = tasks.filter(t => t.priority === "High").length;
let priorityPercent = total === 0 ? 0 : (highPriority / total) * 100;

document.getElementById("priorityMeter").value = priorityPercent;
}
```